Projector-Generated Special Functions II

Cyclotomic Stieltjes Packets, Packet Resultants, Positive Energy Resolvents, Gamma Scaling, and Character Decoherence

Komorebi P.

Abstract

This paper develops a second family of special functions generated by exact packet refinement. The starting point is the Bernoulli–Hurwitz child system introduced in the first paper of the series. A finite cyclotomic transform is applied to the children of one parent packet. The Taylor germ of every non-DC detail channel is then identified with a triangular transform of cyclotomic Fourier packets of generalized Stieltjes constants. For a nonprincipal Dirichlet character, the same transform recovers the derivatives $L^{(m)}(1,\chi)$ from generalized Stieltjes constants at rational arguments.

Two scalar packet invariants are introduced. The holomorphic detail variance equals both the sum of the non-DC channel products and the mean pairwise square difference of the children. The primitive packet resultant compresses the complete Galois orbit of primitive cyclotomic details into one entire scalar obstruction. At positive integer order it takes rational values for rational packet parameters.

The arithmetic half of the paper studies the positive squarefree packet energy

$$E_{a,\beta,\chi}(k) = \sum_{n\geq 1} \frac{\mu(n)^2\,|\chi(n)|^2}{n^{2a}} \left(1 - \frac{1}{n^{\beta}}\right)^{2k}.$$

This sequence is proved to be a Hausdorff moment sequence. All finite differences have alternating sign, every finite Hankel matrix is positive definite, and its generating resolvent is analytic and zero-free on $\mathbb{C} \setminus [1,\infty)$. The same energy is also an exact finite difference of the squarefree zeta ratio $\zeta(s)/\zeta(2s)$ with the Euler factors at the conductor removed.

A complete asymptotic law is proved. After normalization, the active packet coordinate $2k/n^{\beta}$ converges to a Gamma distribution with shape $(2a-1)/\beta$. For characters modulo one fixed modulus, distinct character packets become asymptotically orthogonal with normalized correlation $O(k^{-1/(2\beta)})$. The principal signed Möbius sector remains outside this theorem. In the zeta specialization, its subpolynomial coherence is equivalent to the Riemann hypothesis.

All figures were generated by the accompanying Python program. The release contains 20 vector figures, matching PNG files, exact tables, 61 built-in checks, 18 independent tests, and the complete source listing.

Status and claim boundary

The paper proves the Stieltjes-germ formulas, the character transform at $s = 1$, the detail-variance identity, the primitive-resultant criterion, complete monotonicity, finite Hankel positivity, the zero-free resolvent theorem, the squarefree zeta-ratio formula, the packet-energy asymptotic, the Gamma scaling limit, and the character-decoherence theorem under the hypotheses stated below.

Classical inputs include the Laurent expansion of the Hurwitz zeta function, Dirichlet-character orthogonality, Bernoulli multiplication, the squarefree counting law, partial summation, the method of moments, and the Báez-Duarte criterion. The logarithmic quantum Hamiltonian used in the final part was realized in the 2026 quantum-dynamics work of Wei and collaborators [7].

The phrase "generated special function" refers to the exact packet construction and its proved laws. It does not by itself establish historical priority for every scalar specialization. The principal Möbius coherence statement remains open and is equivalent to RH after normalization by the proved diagonal energy law.

Contents

Status and claim boundary 1

I Cyclotomic germs of Bernoulli refinement 5

1 Bernoulli–Hurwitz children 5

2 Cyclotomic Stieltjes packet theorem 7

3 Character packets and derivatives at one 9

II Scalar invariants of a complete refinement 10

4 Holomorphic detail variance 11

5 Bernoulli–Euler and three-channel examples 12

6 Primitive packet resultants 14

III Positive arithmetic packet functions 15

7 The squarefree packet energy 15

8 Hankel positivity and the zero-free resolvent 16

9 A finite-difference zeta-ratio coordinate system 18

IV Asymptotic packet geometry 20

10 The diagonal energy law 20

11 Universal Gamma scaling 22

12 Character packet decoherence **24**

V Quantum realization and the remaining principal obstruction **26**

13 A logarithmic packet Hamiltonian **26**

14 The principal Möbius coherence problem **28**

15 Research questions generated by the calculus **30**

A Formula registry **31**

B Numerical certificate tables **32**

C Implementation map **33**

D Complete Python source **33**

E Reproduction **47**

List of Figures

1.1 The second special-function construction begins with one Bernoulli parent and its exact children. The finite child transform produces local Stieltjes packets, scalar detail invariants, and a positive arithmetic energy family. 6

2.1 The real and imaginary parts of the first cyclotomic Stieltjes packets for a four-channel refinement. The channels contain distinct local Taylor data even though they reconstruct one Bernoulli parent. 8

2.2 The first two-channel detail and its tangent at the removable point. The linear coefficient is the exact constant $-(\log 2)/\sqrt{2}$. 9

3.1 The nonprincipal character modulo four gives the same values from generalized Stieltjes packets and from derivatives of the Dirichlet beta function at one. The visible numerical differences come from independent differentiation of the L-function implementation. . . . 10

4.1 The same refinement viewed in child coordinates and cyclotomic coordinates. The non-DC transform channels contain exactly the scalar variance lost by replacing all children by their parent mean. 12

5.1 The nonnegative two-channel variance for three even Bernoulli orders. The Euler polynomial controls the complete detail energy of the Bernoulli split. 13

5.2 The three exact order-four Bernoulli children. Two children coincide, while the remaining difference produces the rational detail variance $32/27$. 14

6.1 Primitive packet resultants at integer orders for three cyclotomic depths. The plotted scalar is a Galois-invariant compression of all primitive detail channels. 15

7.1 The first nine alternating finite-difference layers of the principal packet energy. Every displayed quantity is nonnegative, as required by the exact Hausdorff moment representation. 16

8.1 The largest and smallest eigenvalues of finite packet-energy Hankel matrices. The tiny negative values at the largest sizes are at floating-point roundoff. The exact Gram representation is strictly positive. 17

8.2 The magnitude of the principal packet-energy resolvent on the slit plane. The excluded positive-real cut contains the poles and their accumulation at one. The theorem rules out zeros everywhere else. 18

9.1 Direct squarefree summation and the exact finite difference of $\zeta(s)/\zeta(2s)$ agree. The small displayed difference is only the truncation error of the direct numerical sum. 20

10.1 The scaled principal energy approaches the exact constant $3/(2^{5/2}\pi^{3/2})$. The final relative error in the finite computation is about 0.16 percent. 22

11.1 The empirical active-packet distributions approach the Gamma$(3/2, 1)$ density. The horizontal coordinate is the intrinsic inverse scale $U_k = 2k/n^2$. 23

11.2 The weighted distribution approaches its Gamma limit as the packet order increases. The plotted distance is a finite numerical certificate rather than the proof, which follows from the moment asymptotics. 24

12.1 The absolute normalized Gram matrix for the four characters modulo five at two packet orders. The diagonal remains one while the off-diagonal channels decay. 25

12.2 The largest off-diagonal correlation in the character packet bank. The theorem gives the general $k^{-1/(2\beta)}$ upper rate. The finite values need not be monotone at every intermediate order. 26

13.1 The positive and signed quantum packet channels. The logarithmic Hamiltonian supplies real-time phases. The scale Hamiltonian localizes the active number packets. 28

14.1 The exact boundary between the proved positive theory and the open signed theory. The positive energy and cross-character layers are controlled. The principal Möbius coherence remains open. 29

15.1 The analytic and arithmetic branches of the second generated family. Cyclotomic germs, positive moments, Gamma scaling, and character Gram matrices now belong to one exact refinement system. 31

Part I

Cyclotomic germs of Bernoulli refinement

Chapter 1

Bernoulli–Hurwitz children

The preceding paper introduced the entire Bernoulli interpolation

$$\mathfrak{B}(z;a) = -z\zeta(1-z,a)$$

and the exact packet family

$$\mathfrak{B}_{M,r}(z;a) = -zM^{z-1}\zeta\left(1-z,\frac{a+r}{M}\right).$$

The reconstruction and refinement laws are

$$\mathfrak{B}(z;a) = \sum_{r=0}^{M-1}\mathfrak{B}_{M,r}(z;a)$$

and

$$\mathfrak{B}_{M,r}(z;a) = \sum_{\ell=0}^{L-1}\mathfrak{B}_{LM,r+\ell M}(z;a).$$

These formulas were proved in [1]. The present paper asks what local and scalar information is created by one refinement step.

Fix integers $M \geq 1$, $L \geq 2$, and $0 \leq r < M$. Set

$$Q = LM, \qquad q_\ell = \frac{a + r + \ell M}{LM}, \qquad 0 \leq \ell < L.$$

The children of the parent packet are

$$b_\ell(z; a) = \mathfrak{B}_{Q, r+\ell M}(z; a) = -zQ^{z-1}\zeta(1 - z, q_\ell).$$

Let

$$\omega_L = \mathrm{e}^{2\pi \mathrm{i}/L}.$$

The unitary child transform is

$$\boxed{W_j(z; a) = \frac{1}{\sqrt{L}} \sum_{\ell=0}^{L-1} \omega_L^{-j\ell} b_\ell(z; a), \qquad 0 \leq j < L.}$$

The DC channel is the parent packet divided by $\sqrt{L}$:

$$W_0(z; a) = \frac{1}{\sqrt{L}} \mathfrak{B}_{M,r}(z; a).$$

The remaining channels record the exact detail created by the refinement.

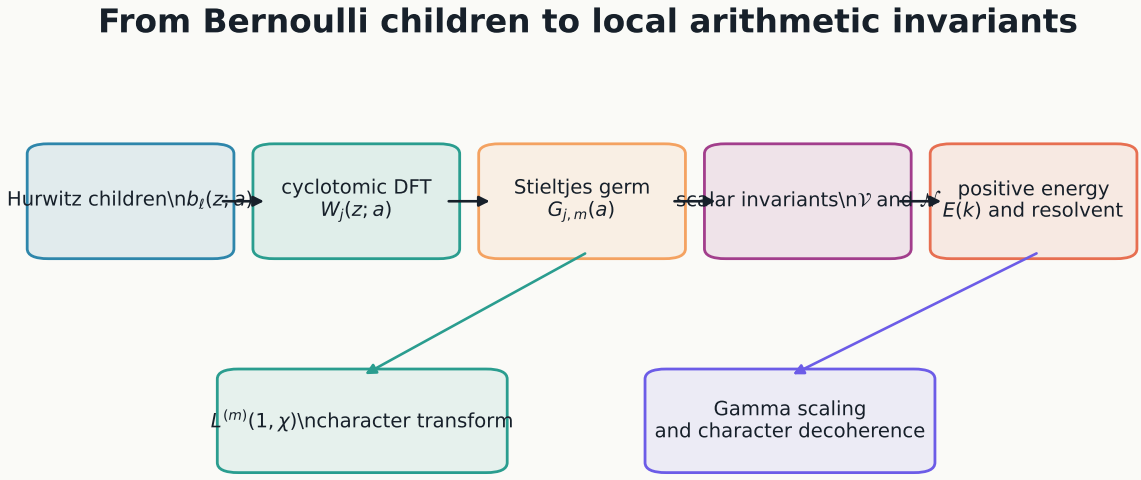


Figure 1.1: The second special-function construction begins with one Bernoulli parent and its exact children. The finite child transform produces local Stieltjes packets, scalar detail invariants, and a positive arithmetic energy family.

Chapter 2

Cyclotomic Stieltjes packet theorem

The generalized Stieltjes constants are defined by the Laurent expansion

$$\zeta(s,q) = \frac{1}{s-1} + \sum_{m=0}^{\infty} \frac{(-1)^m}{m!}\gamma_m(q)(s-1)^m.$$

Substituting $s = 1 - z$ yields

$$\zeta(1-z,q) = -\frac{1}{z} + \sum_{m=0}^{\infty} \gamma_m(q)\frac{z^m}{m!}.$$

Definition 2.1 (Cyclotomic Stieltjes packet)**.** For $0 \le j < L$ and $m \ge 0$, define

$$\boxed{G_{j,m}(a) = \frac{1}{\sqrt{L}} \sum_{\ell=0}^{L-1} \omega_L^{-j\ell}\gamma_m(q_\ell).}$$

Theorem 2.2 (Complete local packet expansion)**.** *For every channel j,*

$$\boxed{W_j(z;a) = \frac{\mathrm{e}^{z\log Q}}{Q}\left[\sqrt{L}\,\delta_{j0} - \sum_{m=0}^{\infty} G_{j,m}(a)\frac{z^{m+1}}{m!}\right].}$$

In particular, every non-DC detail has a zero at the removable Bernoulli point:

$$\boxed{W_j(0;a) = 0 \qquad (j \ne 0).}$$

For $j \ne 0$ and $n \ge 1$,

$$\boxed{W_j^{(n)}(0;a) = -\frac{n}{Q}\sum_{m=0}^{n-1}\binom{n-1}{m}(\log Q)^{n-1-m}G_{j,m}(a).}$$

Proof. Insert the Laurent expansion into the child definition:

$$\begin{aligned} b_\ell(z;a) &= -zQ^{z-1}\left[-\frac{1}{z} + \sum_{m\ge 0}\gamma_m(q_\ell)\frac{z^m}{m!}\right] \\ &= \frac{\mathrm{e}^{z\log Q}}{Q}\left[1 - \sum_{m\ge 0}\gamma_m(q_\ell)\frac{z^{m+1}}{m!}\right]. \end{aligned}$$

Apply the finite Fourier transform over ℓ. The transform of the constant vector is $\sqrt{L}$ in the DC channel and zero in every detail channel. This proves the first formula and the zero at $z = 0$.

For a non-DC channel, multiply the two power series

$$\mathrm{e}^{z\log Q} \quad \text{and} \quad -\sum_{m\ge 0} G_{j,m}(a)\frac{z^{m+1}}{m!}.$$

The coefficient of z^n is

$$-\frac{1}{Q}\sum_{m=0}^{n-1}\frac{G_{j,m}(a)}{m!}\frac{(\log Q)^{n-1-m}}{(n-1-m)!}.$$

Multiplication by $n!$ gives the derivative formula. □

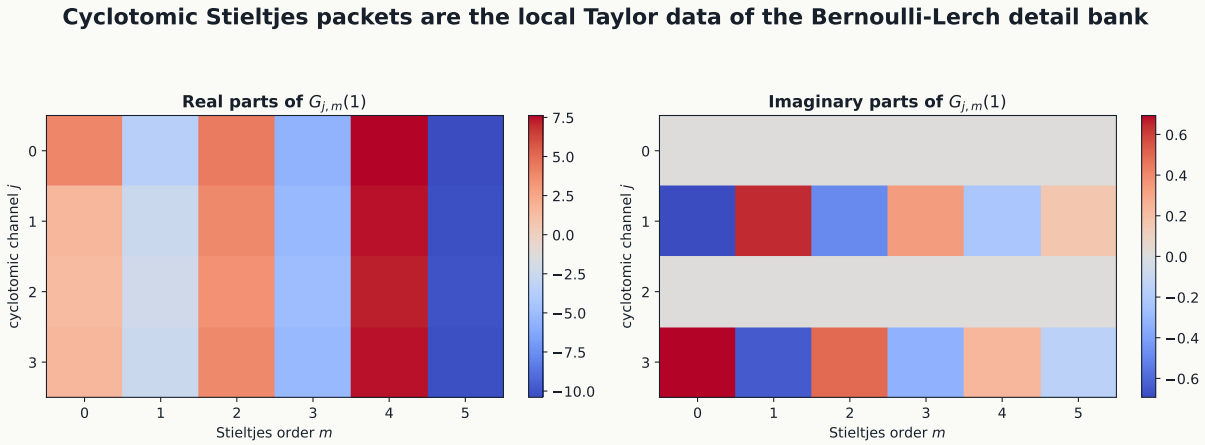


Figure 2.1: The real and imaginary parts of the first cyclotomic Stieltjes packets for a four-channel refinement. The channels contain distinct local Taylor data even though they reconstruct one Bernoulli parent.

Worked example 2.3 (The first Bernoulli–Euler detail coefficient)**.** Take $L = 2$, $M = 1$, $r = 0$, and $a = 1$. Then

$$q_0 = \frac{1}{2}, \qquad q_1 = 1.$$

The first non-DC packet is

$$G_{1,0}(1) = \frac{\gamma_0(1/2) - \gamma_0(1)}{\sqrt{2}}.$$

Since $\gamma_0(q) = -\psi(q)$ and

$$\psi(1/2) = -\gamma - 2\log 2, \qquad \psi(1) = -\gamma,$$

we have

$$G_{1,0}(1) = \sqrt{2}\log 2.$$

The derivative theorem gives

$$\boxed{W_1'(0;1) = -\frac{\log 2}{\sqrt{2}}.}$$

The first four derivatives computed from the packet formula and from direct high-precision differentiation are shown below.

n	packet formula	direct derivative	absolute error
1	-0.490129071734	-0.490129071734	2.62e-62
2	0.226088771874	0.226088771874	5.55e-17
3	0.138676210506	0.138676210506	2.27e-62
4	0.026626672189	0.026626672189	1.11e-16

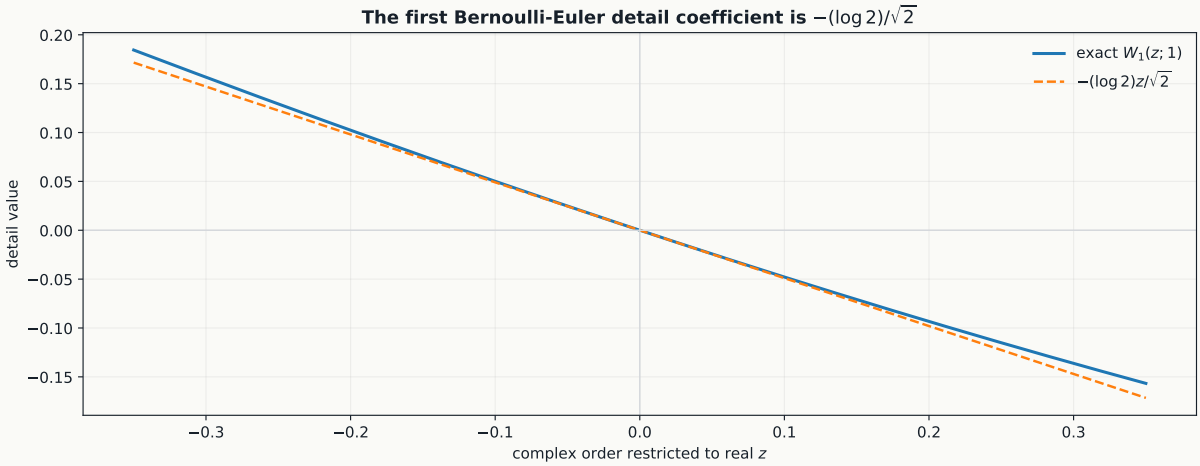


Figure 2.2: The first two-channel detail and its tangent at the removable point. The linear coefficient is the exact constant $-(\log 2)/\sqrt{2}$.

Chapter 3

Character packets and derivatives at one

The additive child transform has a multiplicative counterpart. Let χ be a nonprincipal Dirichlet character modulo q. The classical Hurwitz representation is

$$q^s L(s,\chi) = \sum_{r=1}^{q} \chi(r)\zeta\left(s, \frac{r}{q}\right).$$

The pole terms cancel because

$$\sum_{r=1}^{q} \chi(r) = 0.$$

Definition 3.1 (Character Stieltjes packet)**.** Define

$$S_m(\chi) = \sum_{r=1}^{q} \chi(r)\gamma_m\left(\frac{r}{q}\right).$$

Theorem 3.2 (Character transform at $s = 1$)**.** *For every $m \geq 0$,*

$$\boxed{S_m(\chi) = (-1)^m q \sum_{h=0}^{m} \binom{m}{h} (\log q)^{m-h} L^{(h)}(1,\chi).}$$

Proof. Write $s = 1 + w$. The pole-free left side has expansion

$$\sum_{m\geq 0} \frac{(-1)^m}{m!} S_m(\chi) w^m.$$

The right side is

$$q e^{w \log q} \sum_{h\geq 0} \frac{L^{(h)}(1,\chi)}{h!} w^h.$$

Equate the coefficient of w^m and multiply by $m!$. □

The first two cases are

$$\boxed{\sum_{r=1}^{q} \chi(r)\gamma_0\left(\frac{r}{q}\right) = qL(1,\chi)}$$

and

$$\boxed{\sum_{r=1}^{q} \chi(r)\gamma_1\left(\frac{r}{q}\right) = -q\left[L'(1,\chi) + (\log q)L(1,\chi)\right].}$$

The packet family therefore joins the special values $L(1-n,\chi)$, carried by the integer Bernoulli samples, to the local derivatives $L^{(m)}(1,\chi)$, carried by the Stieltjes germ.

m	Stieltjes packet sum	L-derivative formula	error
0	3.1415926536	3.1415926536	0
1	-5.1267774478	-5.1267774482	4.31e-10
2	7.5603277998	7.5603278010	1.19e-09
3	-10.6337651115	-10.6337651140	2.48e-09

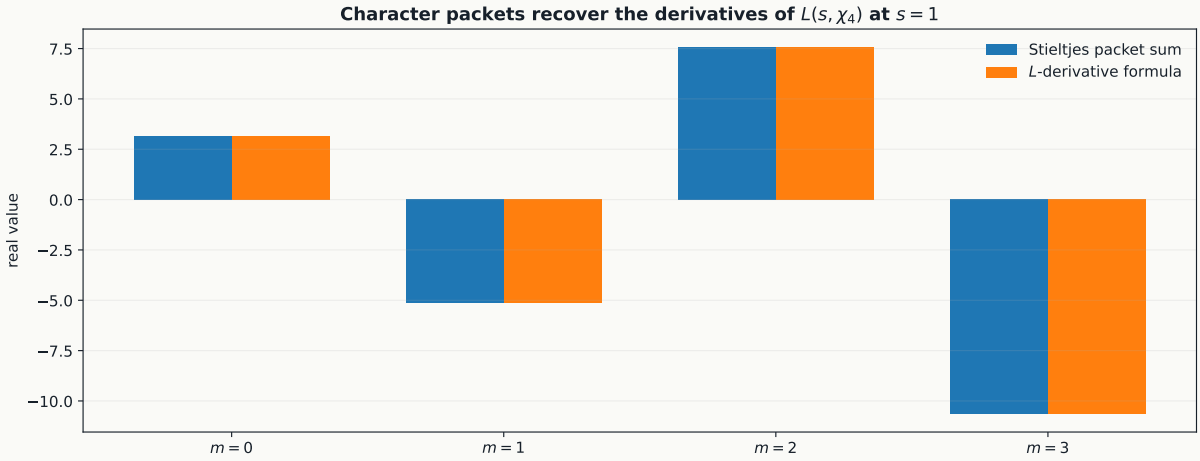


Figure 3.1: The nonprincipal character modulo four gives the same values from generalized Stieltjes packets and from derivatives of the Dirichlet beta function at one. The visible numerical differences come from independent differentiation of the L-function implementation.

Part II

Scalar invariants of a complete refinement

Chapter 4

Holomorphic detail variance

Let $b_0, \dots, b_{L-1}$ be the child packets at fixed (z, a) and let W_j be their normalized finite Fourier transform.

Definition 4.1 (Holomorphic detail variance)**.** Define

$$\boxed{\mathcal{V}_L = \sum_{\ell=0}^{L-1} b_\ell^2 - \frac{1}{L}\left(\sum_{\ell=0}^{L-1} b_\ell\right)^2.}$$

Theorem 4.2 (Three exact forms)**.** *The detail variance satisfies*

$$\boxed{\mathcal{V}_L = \sum_{j=1}^{L-1} W_j W_{L-j}}$$

and

$$\boxed{\mathcal{V}_L = \frac{1}{L} \sum_{0 \le \ell < m < L} (b_\ell - b_m)^2.}$$

For real z and real a, the children are real and

$$\boxed{\mathcal{V}_L = \sum_{j=1}^{L-1} |W_j|^2 \ge 0.}$$

It vanishes precisely when every child value is equal.

Proof. Fourier orthogonality gives

$$\sum_{j=0}^{L-1} W_j W_{L-j} = \sum_{\ell=0}^{L-1} b_\ell^2.$$

The DC term is

$$W_0^2 = \frac{1}{L}\left(\sum_{\ell} b_\ell\right)^2.$$

Subtracting it proves the first identity. Expanding all pairwise squares gives

$$\sum_{\ell<m} (b_\ell - b_m)^2 = L \sum_{\ell} b_\ell^2 - \left(\sum_{\ell} b_\ell\right)^2,$$

which proves the second identity. For real children, $W_{L-j} = \overline{W_j}$, so the first identity becomes a sum of squares. □

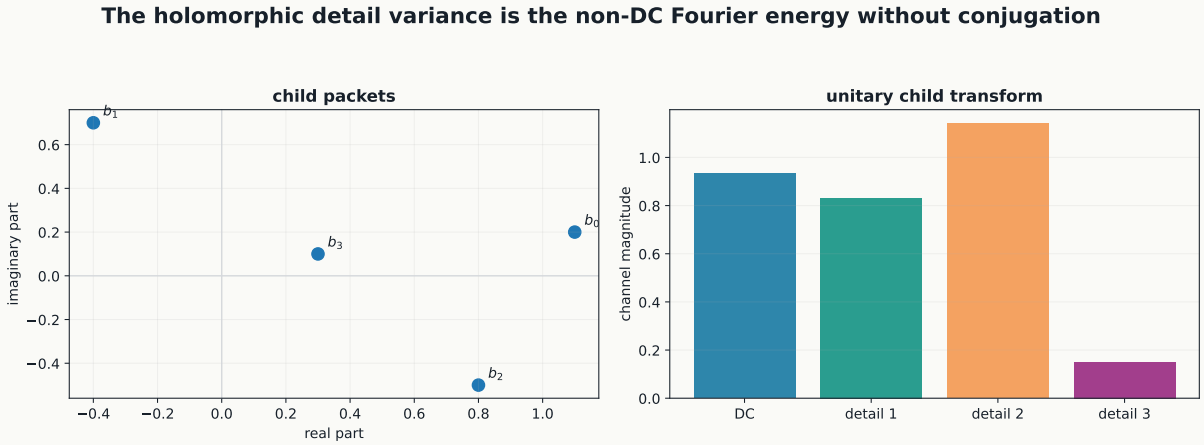


Figure 4.1: The same refinement viewed in child coordinates and cyclotomic coordinates. The non-DC transform channels contain exactly the scalar variance lost by replacing all children by their parent mean.

Chapter 5

Bernoulli–Euler and three-channel examples

Proposition 5.1 (Euler detail as variance square root)**.** *In the two-channel refinement of the Bernoulli parent,*

$$W_1(n;a) = -\frac{n}{2\sqrt{2}} E_{n-1}(a).$$

Consequently,

$$\boxed{\mathcal{V}_2(n;a) = \frac{n^2}{8} E_{n-1}(a)^2.}$$

Proof. The two children at integer order are

$$b_0 = 2^{n-1} B_n(a/2), \qquad b_1 = 2^{n-1} B_n((a+1)/2).$$

The standard Bernoulli–Euler relation gives

$$b_0 - b_1 = -\frac{n}{2} E_{n-1}(a).$$

Divide by $\sqrt{2}$ to obtain W_1. Since $\mathcal{V}_2 = W_1^2$, the variance formula follows. □

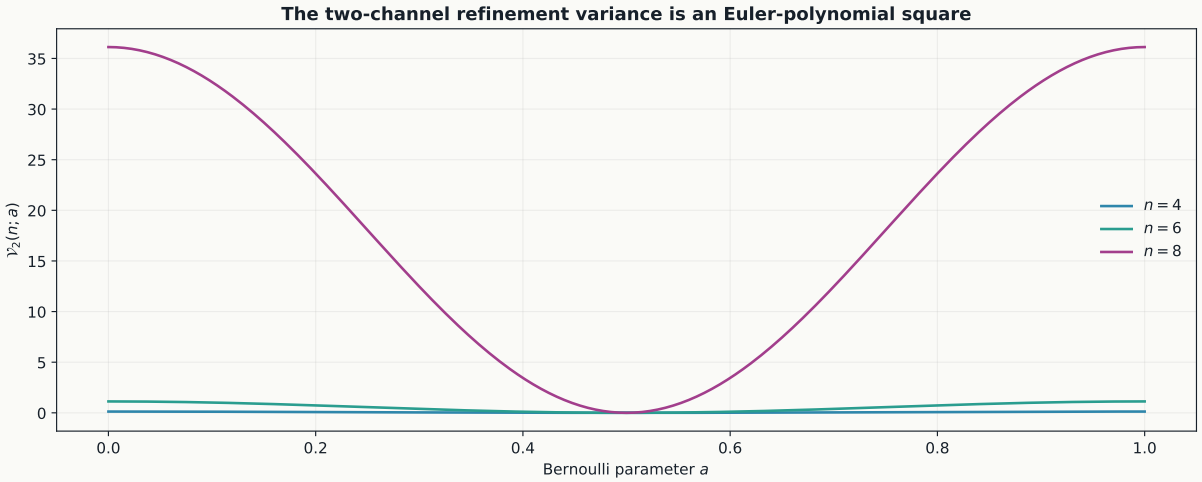


Figure 5.1: The nonnegative two-channel variance for three even Bernoulli orders. The Euler polynomial controls the complete detail energy of the Bernoulli split.

Worked example 5.2 (A three-channel rational variance)**.** Take $L = 3$, $M = 1$, $r = 0$, $a = 0$, and $n = 4$. The exact child values are

$$(b_0, b_1, b_2) = \left(-\frac{9}{10}, \frac{13}{30}, \frac{13}{30}\right).$$

Therefore

$$\begin{aligned}\mathcal{V}_3 &= \frac{1}{3}\left[\left(-\frac{4}{3}\right)^2 + 0^2 + \left(\frac{4}{3}\right)^2\right] \\ &= \boxed{\frac{32}{27}}.\end{aligned}$$

The two primitive channels are conjugate and satisfy

$$W_1 W_2 = \frac{16}{27}.$$

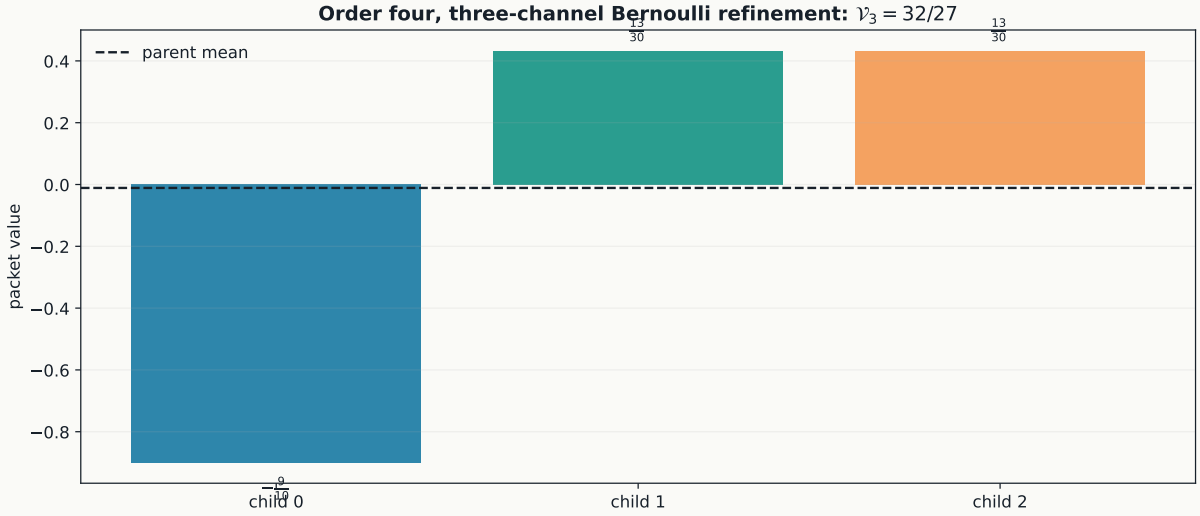


Figure 5.2: The three exact order-four Bernoulli children. Two children coincide, while the remaining difference produces the rational detail variance 32/27.

Chapter 6

Primitive packet resultants

Define the child polynomial

$$P_{M,r}^{(L)}(X; z, a) = \sum_{\ell=0}^{L-1} b_\ell(z; a) X^\ell.$$

At a root of unity,

$$P_{M,r}^{(L)}(\omega_L^{-j}; z, a) = \sqrt{L}\, W_j(z; a).$$

Definition 6.1 (Primitive packet resultant)**.** Let Φ_L be the Lth cyclotomic polynomial. Define

$$\boxed{\mathcal{N}_{M,r}^{(L)}(z; a) = \mathrm{Res}_X\left(\Phi_L(X), P_{M,r}^{(L)}(X; z, a)\right).}$$

Equivalently,

$$\mathcal{N}_{M,r}^{(L)}(z; a) = \prod_{\substack{1 \le j \le L \\ (j,L)=1}} P_{M,r}^{(L)}(\omega_L^j; z, a).$$

Theorem 6.2 (Primitive collapse criterion)**.** *The primitive resultant is entire in z. It vanishes exactly when at least one primitive cyclotomic detail channel vanishes. If $a \in \mathbb{Q}$ and $n \in \mathbb{N}$, then*

$$\boxed{\mathcal{N}_{M,r}^{(L)}(n; a) \in \mathbb{Q}.}$$

Proof. The resultant is a polynomial in the coefficients $b_\ell(z; a)$, so it is entire. Since Φ_L is monic, the product formula for the resultant is the product of P over all primitive Lth roots. Therefore it is zero exactly when one primitive evaluation is zero. At integer order and rational a, every child equals a rational multiple of a Bernoulli-polynomial value at a rational point, hence is rational. The resultant of two rational polynomials is rational. □

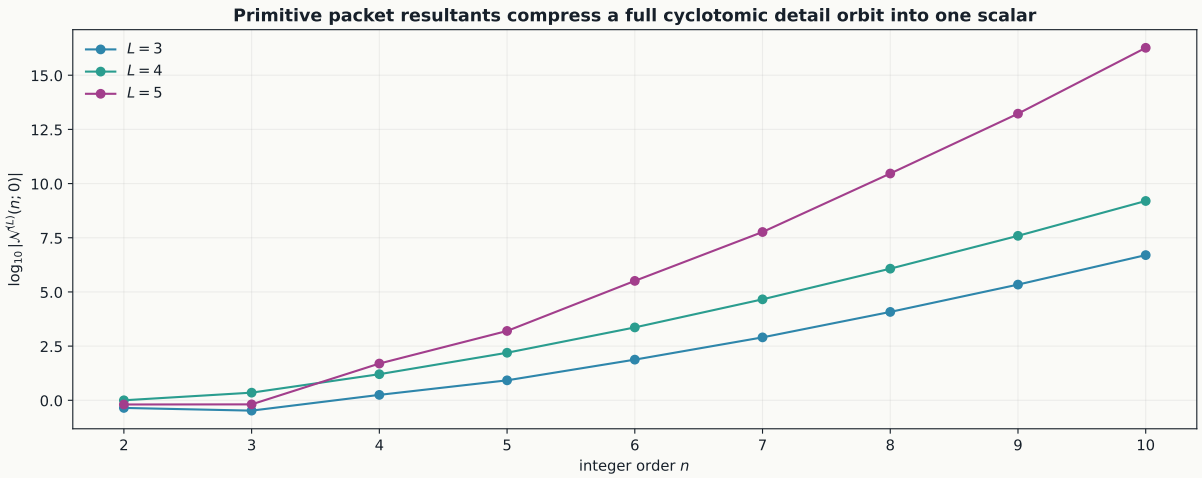


Figure 6.1: Primitive packet resultants at integer orders for three cyclotomic depths. The plotted scalar is a Galois-invariant compression of all primitive detail channels.

Part III

Positive arithmetic packet functions

Chapter 7

The squarefree packet energy

Let $a > 1/2$, $\beta > 0$, and let χ be a Dirichlet character modulo q. Define

$$\boxed{E_{a,\beta,\chi}(k) = \sum_{n=1}^{\infty} \frac{\mu(n)^2\,|\chi(n)|^2}{n^{2a}} \left(1 - \frac{1}{n^{\beta}}\right)^{2k}.}$$

Set

$$w_n = \frac{\mu(n)^2\,|\chi(n)|^2}{n^{2a}}, \qquad x_n = \left(1 - \frac{1}{n^{\beta}}\right)^2.$$

Then

$$E_{a,\beta,\chi}(k) = \sum_n w_n x_n^k.$$

The support lies in $[0, 1)$ and the total mass is finite because $a > 1/2$.

Theorem 7.1 (Complete monotonicity)**.** *For all integers $m, k \geq 0$,*

$$\boxed{(-1)^m \Delta^m E_{a,\beta,\chi}(k) = \sum_n w_n x_n^k (1 - x_n)^m \geq 0.}$$

Thus $E_{a,\beta,\chi}$ is a Hausdorff moment sequence. In particular it is decreasing and log-convex:

$$E(k)^2 \leq E(k-1)E(k+1).$$

Proof. Since $\Delta x^k = x^k(x-1)$, repeated finite differences give

$$\Delta^m x^k = x^k(x-1)^m.$$

Sum against the nonnegative weights w_n. Log-convexity is the Cauchy–Schwarz inequality applied to $x_n^{(k-1)/2}$ and $x_n^{(k+1)/2}$ in the weighted sequence space. □

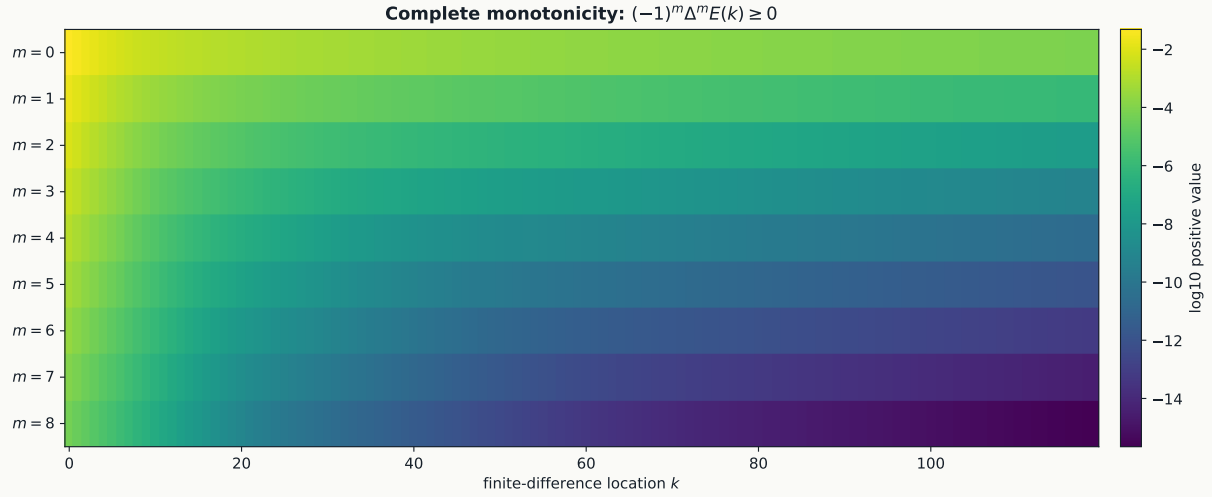


Figure 7.1: The first nine alternating finite-difference layers of the principal packet energy. Every displayed quantity is nonnegative, as required by the exact Hausdorff moment representation.

Chapter 8

Hankel positivity and the zero-free resolvent

Theorem 8.1 (Strict finite Hankel positivity)**.** *For every $d \geq 0$, the Hankel matrix*

$$H_d = (E(i+j))_{0 \leq i,j \leq d}$$

is positive definite.

Proof. For a nonzero vector $c = (c_0, \ldots, c_d)$,

$$\begin{aligned} c^* H_d c &= \sum_{i,j} c_i \overline{c_j} E(i+j) \\ &= \sum_n w_n \left| \sum_{i=0}^{d} c_i x_n^i \right|^2 . \end{aligned}$$

The support contains infinitely many distinct x_n with positive weight. A nonzero polynomial of degree at most d cannot vanish at all of them. Hence the sum is strictly positive. □

The smallest computed eigenvalues eventually reach floating-point roundoff. The theorem is exact and does not depend on those numerical signs.

size	smallest eigenvalue	largest eigenvalue
2	6.345e-04	1.075e-01
3	1.865e-06	1.197e-01
4	6.036e-10	1.253e-01
5	8.948e-14	1.282e-01
6	-2.410e-17	1.297e-01
7	-2.218e-17	1.307e-01
8	-3.034e-17	1.313e-01
9	-3.447e-17	1.317e-01
10	-3.586e-17	1.320e-01

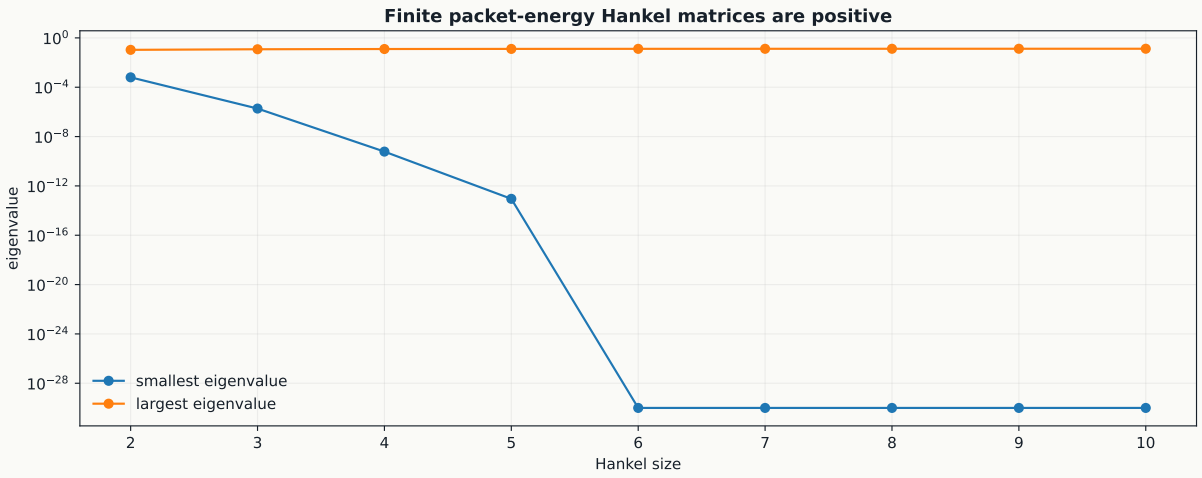


Figure 8.1: The largest and smallest eigenvalues of finite packet-energy Hankel matrices. The tiny negative values at the largest sizes are at floating-point roundoff. The exact Gram representation is strictly positive.

Definition 8.2 (Packet-energy resolvent)**.** Define

$$\boxed{\mathcal{E}_{a,\beta,\chi}(z) = \sum_{k=0}^{\infty} E_{a,\beta,\chi}(k) z^k.}$$

For $|z| < 1$,

$$\mathcal{E}_{a,\beta,\chi}(z) = \sum_{n} \frac{w_n}{1 - z x_n}.$$

Theorem 8.3 (Zero-free slit plane)**.** *The resolvent extends analytically to*

$$\mathbb{C} \setminus [1, \infty)$$

and satisfies

$$\boxed{\mathcal{E}_{a,\beta,\chi}(z) \neq 0 \qquad (z \in \mathbb{C} \setminus [1, \infty)).}$$

Proof. Uniform convergence on compact subsets of the slit plane follows from $\sum_n w_n < \infty$ and a uniform lower bound on $|1 - zx|$ for $x \in [0, 1]$.

For $z \neq 0$, set $u = 1/z$ and

$$S(u) = \sum_n \frac{w_n}{u - x_n}.$$

Then

$$\mathcal{E}(z) = uS(u).$$

If $\Im u > 0$, then

$$\Im S(u) = -\Im u \sum_n \frac{w_n}{|u - x_n|^2} < 0.$$

The sign reverses in the lower half-plane. If $u > 1$, then every summand is positive. If $u < 0$, every summand is negative. These cases cover the image of the slit domain under $u = 1/z$. Therefore $S(u)$ and $\mathcal{E}(z)$ never vanish. At $z = 0$, $\mathcal{E}(0) = \sum_n w_n > 0$. □

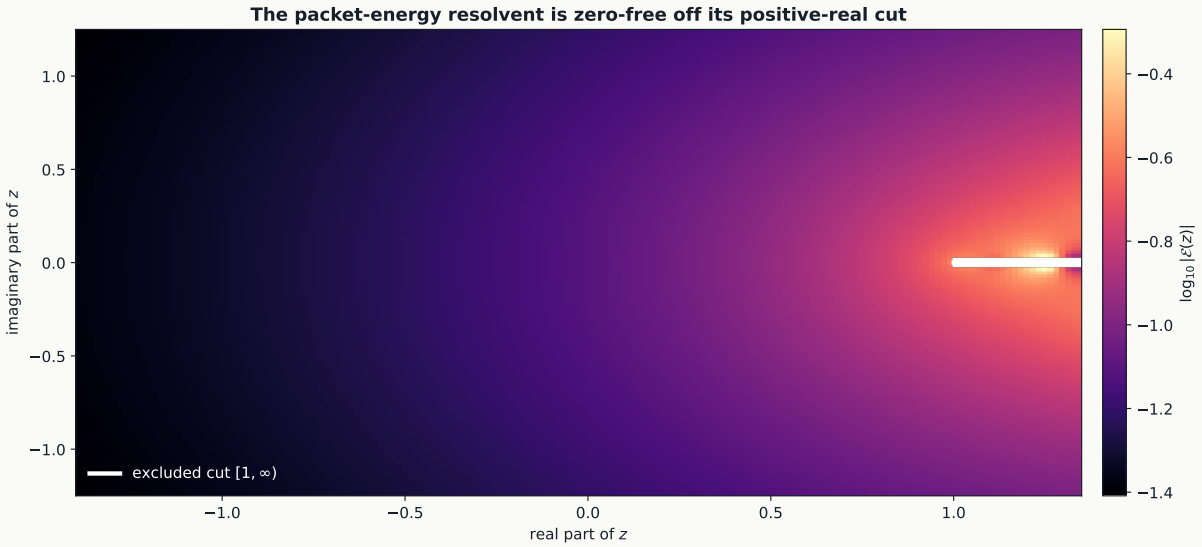


Figure 8.2: The magnitude of the principal packet-energy resolvent on the slit plane. The excluded positive-real cut contains the poles and their accumulation at one. The theorem rules out zeros everywhere else.

Chapter 9

A finite-difference zeta-ratio coordinate system

Because $|\chi(n)|^2$ is the indicator of $(n, q) = 1$, define

$$D_q(s) = \sum_{\substack{n \geq 1 \\ (n,q)=1}} \frac{\mu(n)^2}{n^s}.$$

Its Euler product is

$$\boxed{D_q(s) = \frac{\zeta(s)}{\zeta(2s)} \prod_{p|q} \left(1 + p^{-s}\right)^{-1}.}$$

Theorem 9.1 (Exact zeta-ratio finite difference)**.** *For every integer $k \geq 0$,*

$$\boxed{E_{a,\beta,\chi}(k) = \sum_{j=0}^{2k} (-1)^j \binom{2k}{j} D_q(2a + \beta j).}$$

Proof. Expand the filter by the binomial theorem:

$$\left(1 - n^{-\beta}\right)^{2k} = \sum_{j=0}^{2k} (-1)^j \binom{2k}{j} n^{-\beta j}.$$

Absolute convergence permits interchange with the squarefree Dirichlet series. The inner sum is $D_q(2a + \beta j)$. □

k	direct energy	zeta-ratio difference	truncation error
1	4.7804610e-02	4.7804610e-02	7.494e-16
2	3.0200677e-02	3.0200677e-02	4.406e-16
3	1.9767515e-02	1.9767515e-02	3.227e-16
4	1.3470094e-02	1.3470094e-02	2.481e-16
5	9.5803728e-03	9.5803728e-03	1.665e-16
6	7.1098189e-03	7.1098189e-03	1.639e-16
7	5.4893011e-03	5.4893011e-03	1.266e-16
8	4.3883096e-03	4.3883096e-03	8.153e-17

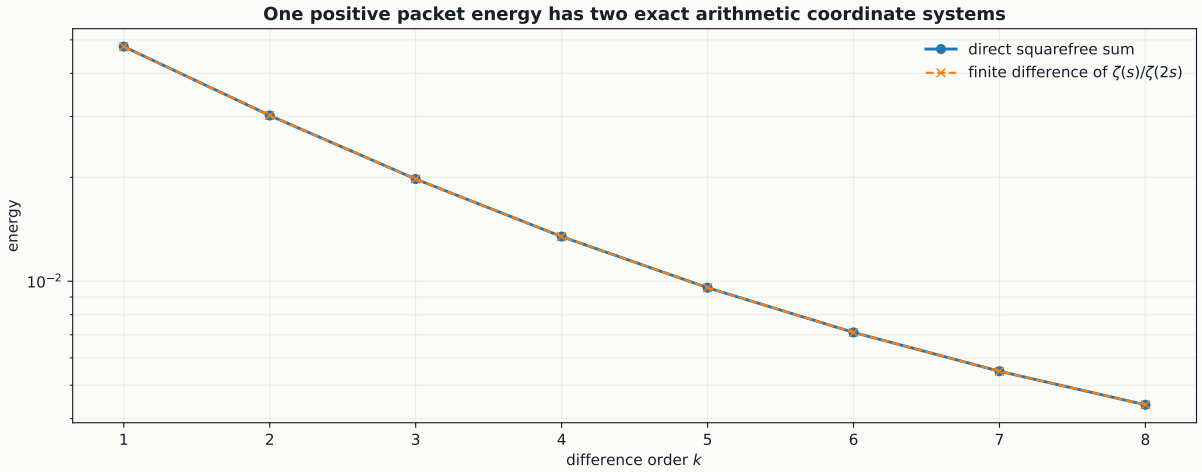


Figure 9.1: Direct squarefree summation and the exact finite difference of $\zeta(s)/\zeta(2s)$ agree. The small displayed difference is only the truncation error of the direct numerical sum.

Part IV

Asymptotic packet geometry

Chapter 10

The diagonal energy law

Let

$$A_q(x) = \sum_{\substack{n \le x \\ (n,q)=1}} \mu(n)^2.$$

The elementary squarefree counting estimate is

$$A_q(x) = \delta_q x + O_q(\sqrt{x}),$$

where

$$\boxed{\delta_q = \frac{1}{\zeta(2)} \prod_{p \mid q} \frac{p}{p+1}.}$$

Set

$$\alpha = \frac{2a-1}{\beta}.$$

Theorem 10.1 (Packet-energy asymptotic)**.** *For $a > 1/2$ and $\beta > 0$,*

$$\boxed{E_{a,\beta,\chi}(k) \sim \frac{\delta_q}{\beta} \Gamma(\alpha)(2k)^{-\alpha}.}$$

More precisely, the error produced by the $O_q(\sqrt{x})$ remainder is

$$O_{a,\beta,q}\left(k^{-(2a-1/2)/\beta}\right).$$

Proof. Use Stieltjes summation with

$$f_k(x) = x^{-2a}(1 - x^{-\beta})^{2k}.$$

The density term contributes

$$\delta_q \int_1^\infty f_k(x)\,\mathrm{d}x.$$

With $y = x^{-\beta}$,

$$\begin{aligned}\int_1^\infty f_k(x)\,\mathrm{d}x &= \frac{1}{\beta}\int_0^1 y^{\alpha-1}(1-y)^{2k}\,\mathrm{d}y \\ &= \frac{1}{\beta}\mathrm{B}(\alpha, 2k+1).\end{aligned}$$

The beta-function asymptotic gives

$$\mathrm{B}(\alpha, 2k+1) \sim \Gamma(\alpha)(2k)^{-\alpha}.$$

Write the squarefree counting remainder as $R_q(x) = O_q(\sqrt{x})$. Partial summation bounds its contribution by a constant times

$$\int_1^\infty \sqrt{x}\,|f_k'(x)|\,\mathrm{d}x.$$

Scaling $x = k^{1/\beta}y$ shows that this integral is

$$O\Big(k^{-(2a-1/2)/\beta}\Big),$$

which is smaller than the main term by $k^{-1/(2\beta)}$. □

For $a = \beta = 2$ and the principal character,

$$\boxed{E_k \sim \frac{3}{2^{5/2}\pi^{3/2}}k^{-3/2}.}$$

k	E_k	$k^{3/2}E_k$	relative error
100	9.397187e-05	0.0939718675	1.332e-02
300	1.844163e-05	0.0958255236	6.143e-03
1000	3.028409e-06	0.0957666957	5.525e-03
3000	5.760934e-07	0.0946618062	6.076e-03
10000	9.539419e-08	0.0953941855	1.614e-03
30000	1.833371e-08	0.0952647535	2.551e-04
100000	3.007053e-09	0.0950913727	1.565e-03

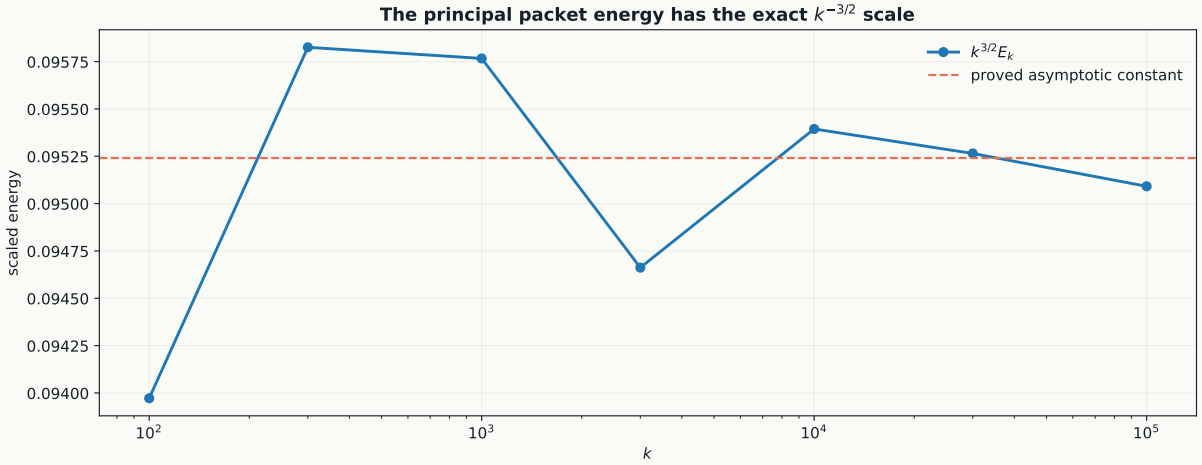


Figure 10.1: The scaled principal energy approaches the exact constant $3/(2^{5/2}\pi^{3/2})$. The final relative error in the finite computation is about 0.16 percent.

Chapter 11

Universal Gamma scaling

Normalize the packet weights into a probability distribution:

$$\mathbb{P}_k(n) = \frac{\mu(n)^2\,|\chi(n)|^2\,n^{-2a}(1-n^{-\beta})^{2k}}{E_{a,\beta,\chi}(k)}.$$

Define

$$\boxed{U_k = \frac{2k}{n^\beta}.}$$

Theorem 11.1 (Gamma scaling limit)**.** *As $k \to \infty$,*

$$\boxed{U_k \Longrightarrow \mathrm{Gamma}(\alpha, 1), \qquad \alpha = \frac{2a-1}{\beta}.}$$

Equivalently, for every bounded continuous φ,

$$\lim_{k\to\infty} \sum_n \mathbb{P}_k(n)\varphi\left(\frac{2k}{n^\beta}\right) = \frac{1}{\Gamma(\alpha)} \int_0^\infty \varphi(u) u^{\alpha-1} \mathrm{e}^{-u}\,\mathrm{d}u.$$

Proof. For every integer $m \geq 0$,

$$\mathbb{E}_k[U_k^m] = (2k)^m \frac{E_{a+\beta m/2,\beta,\chi}(k)}{E_{a,\beta,\chi}(k)}.$$

Apply the packet-energy asymptotic to numerator and denominator. The squarefree density cancels and gives

$$\lim_{k\to\infty} \mathbb{E}_k[U_k^m] = \frac{\Gamma(\alpha+m)}{\Gamma(\alpha)}.$$

These are the moments of $\mathrm{Gamma}(\alpha, 1)$. The Gamma distribution is moment-determinate. The first moments remain bounded, so the family is tight. The method of moments gives weak convergence. □

For the principal RH parameters $a = \beta = 2$,

$$\alpha = \frac{3}{2}.$$

The active packet window therefore has a complete limiting profile, not only the rough location $n \asymp \sqrt{k}$.

k	mean	variance	$q_{0.1}$	$q_{0.5}$	$q_{0.9}$	KS
300	1.54561	1.54638	0.31798	1.18207	3.01649	0.05780
3000	1.50231	1.59381	0.29310	1.17179	3.29253	0.04726
30000	1.49275	1.48729	0.29110	1.20548	3.08603	0.02256
100000	1.50160	1.52410	0.29473	1.16989	3.10136	0.01560

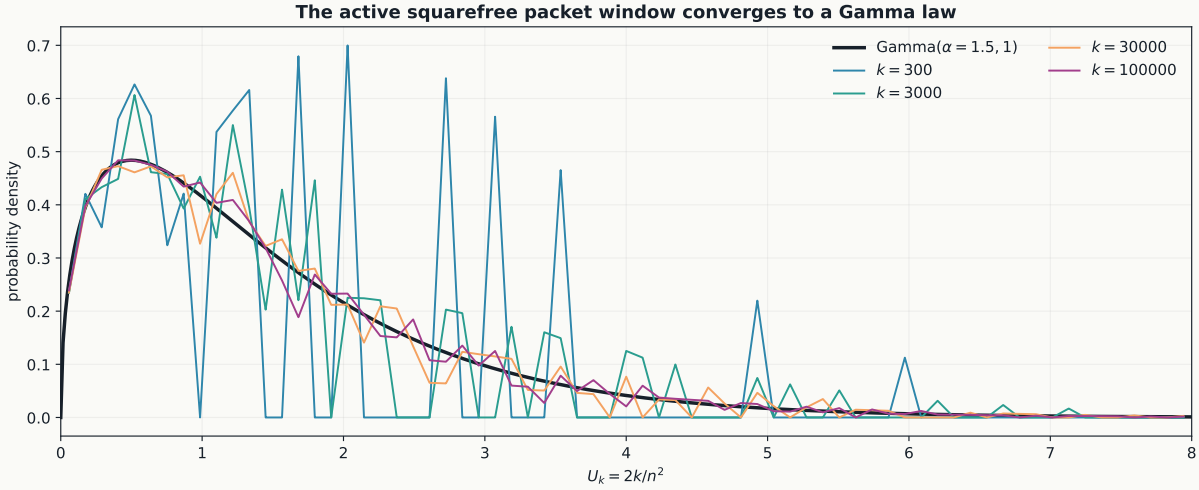


Figure 11.1: The empirical active-packet distributions approach the Gamma$(3/2, 1)$ density. The horizontal coordinate is the intrinsic inverse scale $U_k = 2k/n^2$.

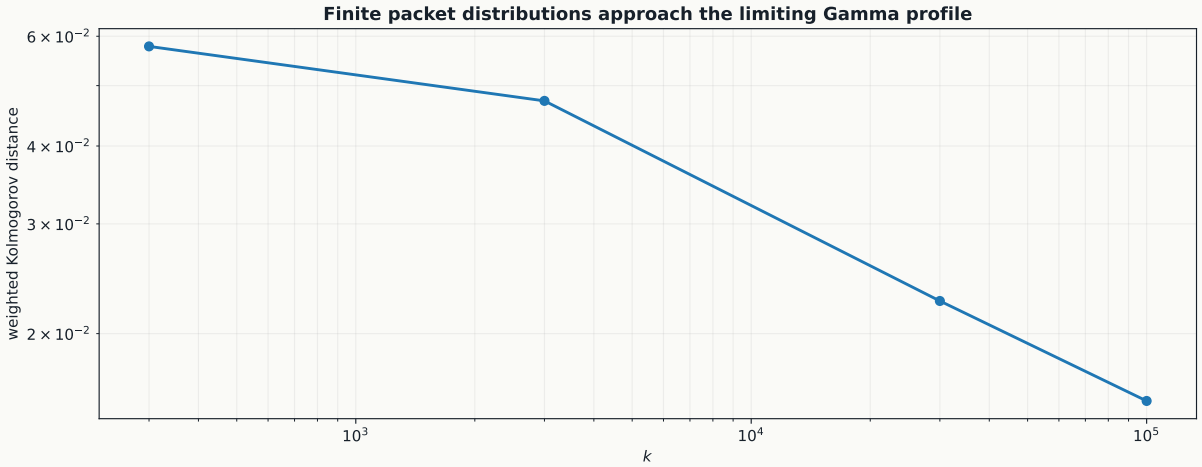


Figure 11.2: The weighted distribution approaches its Gamma limit as the packet order increases. The plotted distance is a finite numerical certificate rather than the proof, which follows from the moment asymptotics.

Chapter 12

Character packet decoherence

Let χ and ψ be Dirichlet characters modulo the same integer q. Define the packet Gram entry

$$\boxed{G_{\chi,\psi}(k) = \sum_{n=1}^{\infty} \frac{\mu(n)^2 \chi(n)\overline{\psi(n)}}{n^{2a}} \left(1 - \frac{1}{n^{\beta}}\right)^{2k}.}$$

The diagonal is the positive packet energy.

Lemma 12.1 (Squarefree character cancellation)**.** *If ξ is a nonprincipal Dirichlet character modulo q, then*

$$\boxed{\sum_{n \le X} \mu(n)^2 \xi(n) = O_q(\sqrt{X}).}$$

Proof. Use

$$\mu(n)^2 = \sum_{d^2 \mid n} \mu(d).$$

Then

$$\sum_{n \le X} \mu(n)^2 \xi(n) = \sum_{d \le \sqrt{X}} \mu(d) \xi(d^2) \sum_{m \le X/d^2} \xi(m).$$

A nonprincipal character has zero sum over a complete period, so every partial sum is $O(q)$. There are $O(\sqrt{X})$ outer terms. □

Theorem 12.2 (Character decoherence)**.** *If $\chi \neq \psi$, then*

$$\boxed{G_{\chi,\psi}(k) = O_{a,\beta,q}\left(k^{-(2a-1/2)/\beta}\right).}$$

Consequently,

$$\boxed{\frac{G_{\chi,\psi}(k)}{\sqrt{G_{\chi,\chi}(k)G_{\psi,\psi}(k)}} = O_{a,\beta,q}\left(k^{-1/(2\beta)}\right).}$$

Proof. The product $\xi = \chi\overline{\psi}$ is nonprincipal. Let

$$A_\xi(X) = \sum_{n\le X} \mu(n)^2\xi(n) = O_q(\sqrt{X}).$$

Apply partial summation to

$$f_k(x) = x^{-2a}(1-x^{-\beta})^{2k}.$$

The resulting bound is a constant times

$$\int_1^\infty \sqrt{x}\,|f_k'(x)|\,\mathrm{d}x = O\Big(k^{-(2a-1/2)/\beta}\Big).$$

The two diagonal entries are asymptotic to positive constants times $k^{-(2a-1)/\beta}$. Dividing gives the normalized rate. □

For $a = \beta = 2$, the normalized decay rate is

$$\boxed{k^{-1/4}.}$$

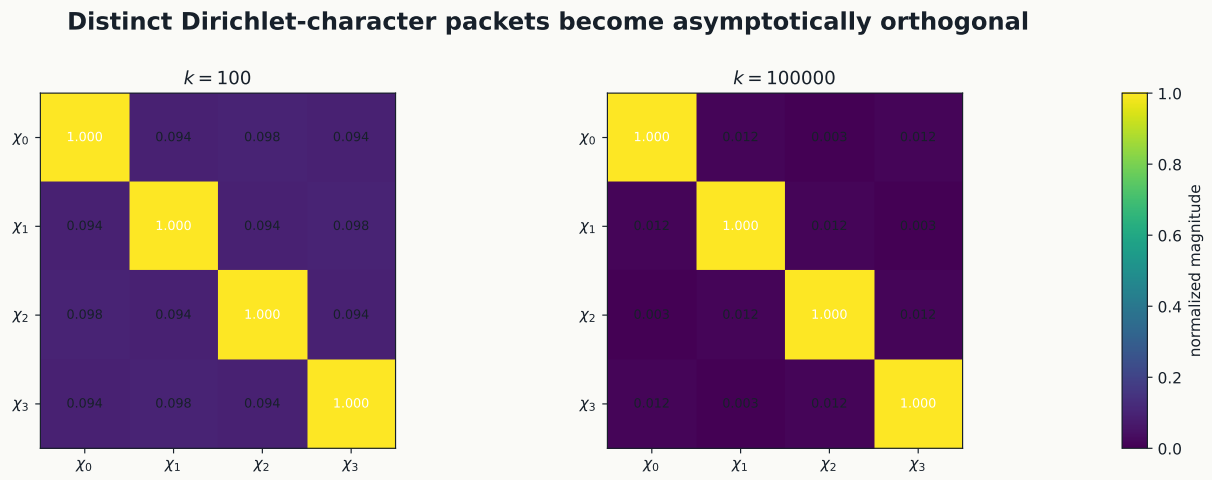


Figure 12.1: The absolute normalized Gram matrix for the four characters modulo five at two packet orders. The diagonal remains one while the off-diagonal channels decay.

k	largest normalized off-diagonal entry
100	0.09807746
300	0.07180785
1000	0.05948305
3000	0.05265039
10000	0.01447532
30000	0.01243919
100000	0.01185877

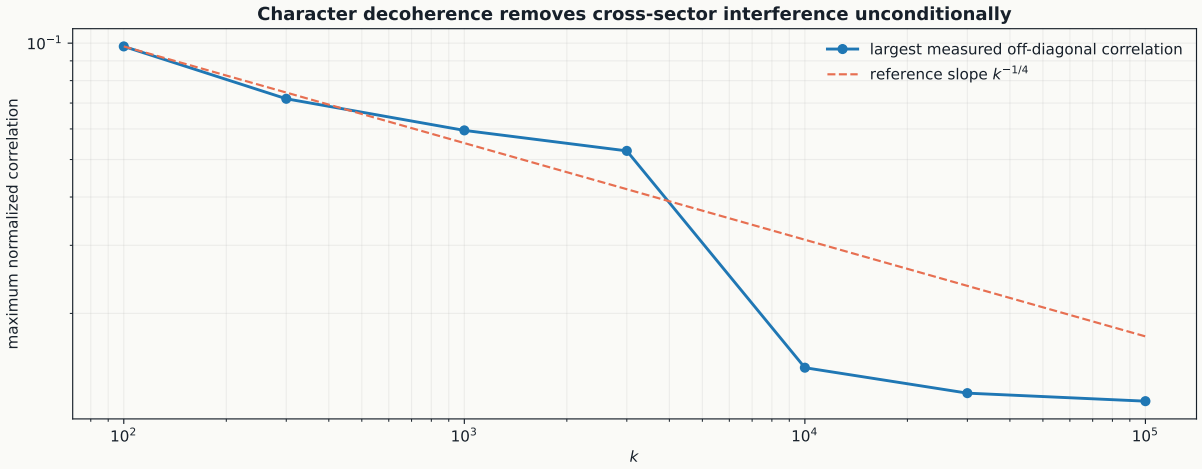


Figure 12.2: The largest off-diagonal correlation in the character packet bank. The theorem gives the general $k^{-1/(2\beta)}$ upper rate. The finite values need not be monotone at every intermediate order.

Part V

Quantum realization and the remaining principal obstruction

Chapter 13

A logarithmic packet Hamiltonian

Work in

$$\mathcal{H} = \ell^2(\{2, 3, \ldots\}).$$

For $a > 1$, define the amplitude vector

$$|\phi_a\rangle = \sum_{n\geq 2} n^{-a/2} |n\rangle .$$

For $a > 1/2$, define the diagonal-power vector

$$|\psi_a\rangle = \sum_{n\geq 2} n^{-a} |n\rangle .$$

Let

$$H |n\rangle = \log n \ |n\rangle ,$$
$$K_\beta |n\rangle = -\log(1 - n^{-\beta}) |n\rangle ,$$

and

$$\mathcal{M}_\chi |n\rangle = \mu(n)\chi(n) |n\rangle .$$

The three operators are diagonal and commute.

Proposition 13.1 (Quantum transfer amplitude)**.** *The generalized signed transfer is*

$$\boxed{\mathcal{C}_{a,\beta,\chi}(k,t) = \langle \phi_a | \, \mathcal{M}_\chi \mathrm{e}^{-kK_\beta} \mathrm{e}^{-\mathrm{i}tH} \, | \phi_a \rangle \, .}$$

Its positive diagonal power is

$$\boxed{E_{a,\beta,\chi}(k) = \langle \psi_a | \, \mathcal{M}_\chi^* \mathcal{M}_\chi \mathrm{e}^{-2kK_\beta} \, | \psi_a \rangle \, .}$$

It is also the long-time mean power of the signed transfer:

$$\boxed{E_{a,\beta,\chi}(k) = \lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} |\mathcal{C}_{a,\beta,\chi}(k,t)|^2 \, \mathrm{d}t.}$$

Proof. The first matrix element has number-basis factors

$$n^{-a}, \qquad \mu(n)\chi(n), \qquad (1-n^{-\beta})^k, \qquad n^{-\mathrm{i}t},$$

which give the signed transfer. The second matrix element has factors

$$n^{-2a}, \qquad \mu(n)^2 \, |\chi(n)|^2 \, , \qquad (1-n^{-\beta})^{2k},$$

which give the positive packet energy. Finally, expand the squared modulus of the transfer. The time average of $\mathrm{e}^{-\mathrm{i}t(\log n - \log m)}$ vanishes when $n \neq m$ and equals one when $n = m$. Only the diagonal terms remain. □

The logarithmic Hamiltonian is not introduced only as formal notation. Wei and collaborators construct a quantum system with spectrum $E_n = \log n$, thermal weights $n^{-\beta}$, and dynamical phases $n^{-\mathrm{i}t}$ [7]. Their observable directly encodes a zeta amplitude. The present packet system uses the same logarithmic spectral coordinate, then adds the squarefree or character sign and the scale filter e^{-kK_β}.

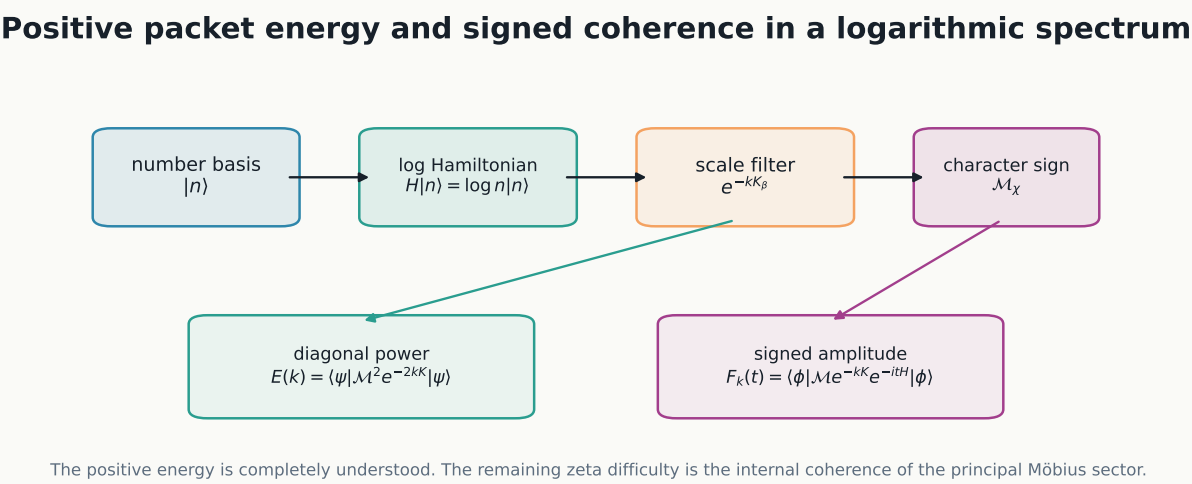


Figure 13.1: The positive and signed quantum packet channels. The logarithmic Hamiltonian supplies real-time phases. The scale Hamiltonian localizes the active number packets.

Chapter 14

The principal Möbius coherence problem

In the principal zeta specialization, set

$$a = 2, \qquad \beta = 2, \qquad \chi = 1.$$

Then

$$c_k = \sum_{n=1}^{\infty} \frac{\mu(n)}{n^2} \left(1 - \frac{1}{n^2}\right)^k$$

which is the Báez-Duarte coefficient. Define

$$\Gamma_k = \frac{|c_k|^2}{E_k}.$$

The diagonal theorem gives

$$E_k \asymp k^{-3/2}.$$

The Báez-Duarte criterion gives

$$\mathrm{RH} \iff c_k = O_\varepsilon(k^{-3/4+\varepsilon}) \quad \text{for every } \varepsilon > 0.$$

Therefore:

Theorem 14.1 (Normalized principal coherence criterion)**.**

$$\boxed{\mathrm{RH} \iff \Gamma_k = O_\varepsilon(k^{\varepsilon}) \quad \textit{for every } \varepsilon > 0.}$$

Proof. If RH holds, square the Báez-Duarte estimate and divide by $E_k \asymp k^{-3/2}$. This gives $\Gamma_k = O_\varepsilon(k^{2\varepsilon})$, which is the stated form after renaming the arbitrary exponent.

Conversely, if $\Gamma_k = O_\varepsilon(k^\varepsilon)$, then

$$|c_k|^2 \ll_\varepsilon k^{-3/2+\varepsilon}.$$

Taking square roots gives the Báez-Duarte estimate after another renaming of the arbitrary exponent. □

The character-decoherence theorem controls interference between distinct character sectors. It does not control the internal Möbius coherence of the principal sector. That remaining signed estimate carries the RH-scale difficulty.

Conjecture 14.2 (Principal packet coherence)**.** *For every $\varepsilon > 0$,*

$$\boxed{\left|\sum_{n=1}^{\infty} \frac{\mu(n)}{n^2}\left(1-\frac{1}{n^2}\right)^k\right|^2 \le C_\varepsilon k^\varepsilon \sum_{n=1}^{\infty} \frac{\mu(n)^2}{n^4}\left(1-\frac{1}{n^2}\right)^{2k}.}$$

This conjecture is equivalent to RH. It is not promoted as a proof.

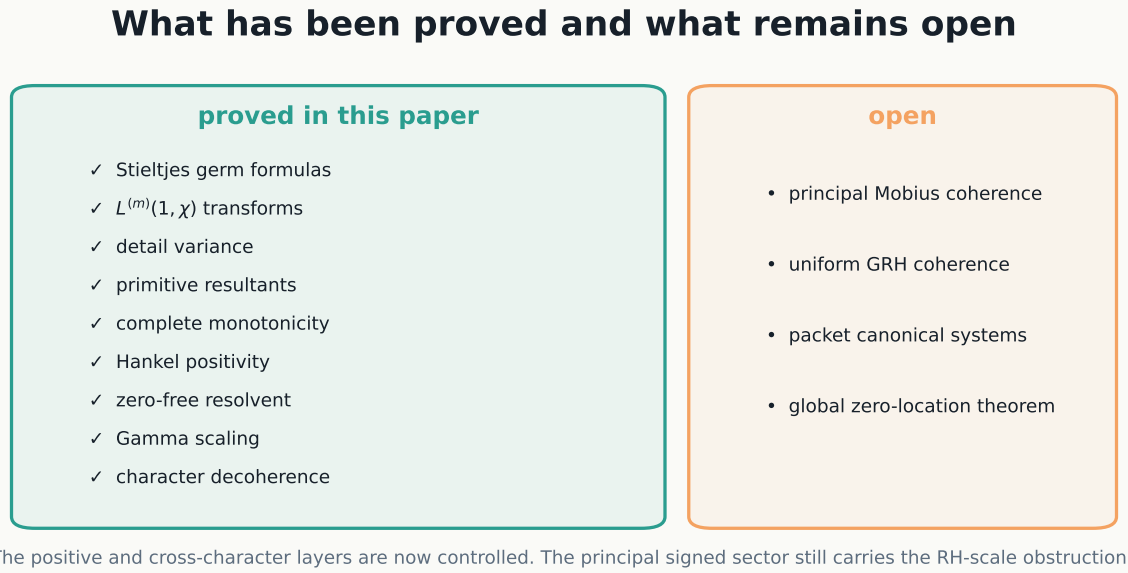


Figure 14.1: The exact boundary between the proved positive theory and the open signed theory. The positive energy and cross-character layers are controlled. The principal Möbius coherence remains open.

Chapter 15

Research questions generated by the calculus

The results above create several concrete questions that are separate from RH.

1. Determine the zero sets and growth order of the primitive packet resultants $\mathcal{N}_{M,r}^{(L)}(z;a)$.
2. Classify the real parameters for which the holomorphic detail variance has only real or nonnegative integer samples.
3. Determine whether the Stieltjes packet matrices $G_{j,m}(a)$ obey finite-rank, total-positivity, or variation-diminishing laws for selected cyclotomic channels.
4. Obtain higher-order asymptotic expansions for $E_{a,\beta,\chi}(k)$ and the Gamma scaling law.
5. Sharpen the elementary $O(k^{-1/(2\beta)})$ character-decoherence rate using stronger character-sum estimates.
6. Construct orthogonal polynomials and continued fractions associated with the positive moment sequence $E(k)$.
7. Determine the spectral measure and boundary behavior of the zero-free resolvent $\mathcal{E}(z)$.
8. Extend the packet Gram theory from Dirichlet characters to automorphic coefficient systems.
9. Decide whether the principal Möbius coherence can be controlled by a local packet inequality rather than a global zero argument.

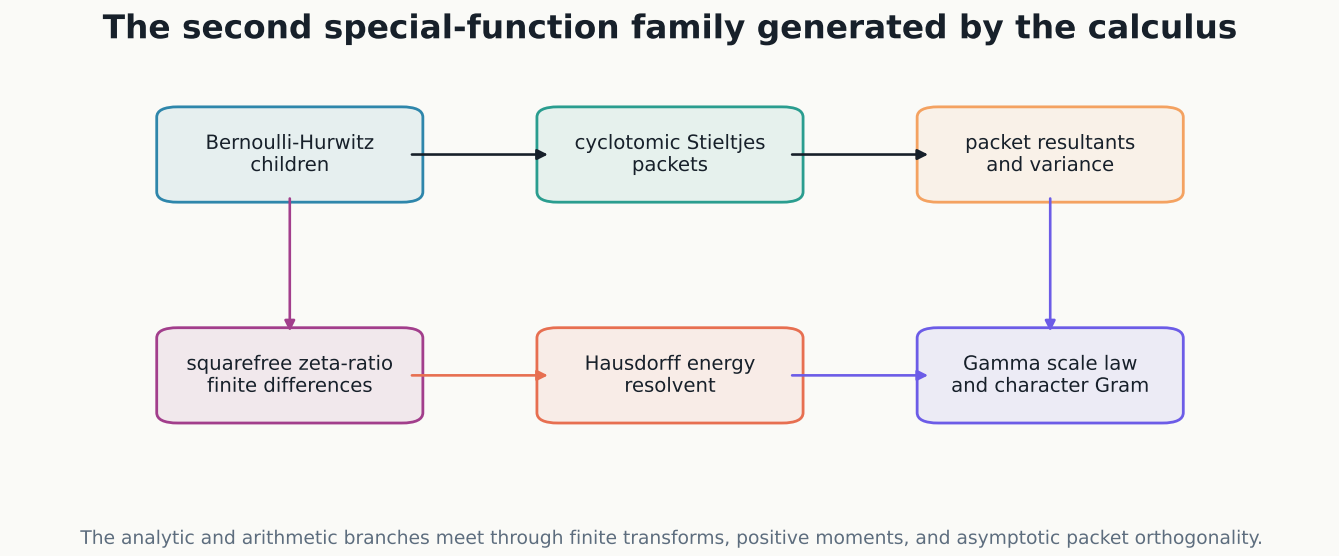


Figure 15.1: The analytic and arithmetic branches of the second generated family. Cyclotomic germs, positive moments, Gamma scaling, and character Gram matrices now belong to one exact refinement system.

Chapter A

Formula registry

Object	Exact formula
Cyclotomic Stieltjes packet	$G_{j,m}(a) = L^{-1/2}\sum_{\ell}\omega_L^{-j\ell}\gamma_m(q_\ell)$
Detail germ	$W_j(z;a) = Q^{-1}\mathrm{e}^{z\log Q}[\sqrt{L}\delta_{j0} - \sum_m G_{j,m}z^{m+1}/m!]$
Character transform	$S_m(\chi) = (-1)^m q\sum_{h\le m}\binom{m}{h}(\log q)^{m-h}L^{(h)}(1,\chi)$
Detail variance	$\mathcal{V}_L = \sum b_\ell^2 - L^{-1}(\sum b_\ell)^2$
Primitive resultant	$\mathcal{N} = \mathrm{Res}(\Phi_L, P)$
Packet energy	$E(k) = \sum_n w_n x_n^k$
Complete monotonicity	$(-1)^m\Delta^m E(k) = \sum_n w_n x_n^k(1-x_n)^m$
Resolvent	$\mathcal{E}(z) = \sum_n w_n/(1-zx_n)$
Zeta-ratio coordinate	$E(k) = \sum_{j=0}^{2k}(-1)^j\binom{2k}{j}D_q(2a+\beta j)$
Energy asymptotic	$E(k) \sim (\delta_q/\beta)\Gamma(\alpha)(2k)^{-\alpha}$
Gamma coordinate	$U_k = 2k/n^\beta \Rightarrow \mathrm{Gamma}(\alpha,1)$
Character decoherence	$\mathrm{Corr}(\chi,\psi) = O(k^{-1/(2\beta)})$ for $\chi\neq\psi$

Chapter B

Numerical certificate tables

Packet-energy asymptotic

k	E_k	$k^{3/2}E_k$	relative error
100	9.397187e-05	0.0939718675	1.332e-02
300	1.844163e-05	0.0958255236	6.143e-03
1000	3.028409e-06	0.0957666957	5.525e-03
3000	5.760934e-07	0.0946618062	6.076e-03
10000	9.539419e-08	0.0953941855	1.614e-03
30000	1.833371e-08	0.0952647535	2.551e-04
100000	3.007053e-09	0.0950913727	1.565e-03

Gamma scaling

k	mean	variance	$q_{0.1}$	$q_{0.5}$	$q_{0.9}$	KS
300	1.54561	1.54638	0.31798	1.18207	3.01649	0.05780
3000	1.50231	1.59381	0.29310	1.17179	3.29253	0.04726
30000	1.49275	1.48729	0.29110	1.20548	3.08603	0.02256
100000	1.50160	1.52410	0.29473	1.16989	3.10136	0.01560

Character decoherence

k	largest normalized off-diagonal entry
100	0.09807746
300	0.07180785
1000	0.05948305
3000	0.05265039
10000	0.01447532
30000	0.01243919
100000	0.01185877

Verification count

61/61 checks passed

The independent test suite contains 18 additional tests. It rederives the main identities without reading the built-in check ledger.

Chapter C

Implementation map

Function	Role
`bernoulli_packet`	Evaluates one Bernoulli–Hurwitz child
`stieltjes_packet`	Computes one cyclotomic Stieltjes packet
`detail_derivative_from_stieltjes`	Builds the local Taylor derivatives
`detail_variance`	Evaluates the scalar non-DC variance
`primitive_packet_resultant_exact`	Computes the exact cyclotomic resultant
`packet_energy_values`	Evaluates the positive squarefree energy
`energy_resolvent`	Evaluates the slit-plane resolvent
`energy_from_zeta_ratio`	Uses the exact finite-difference coordinate
`gamma_weighted_sample`	Builds the active inverse-scale distribution
`character_gram_matrix`	Constructs the finite character packet Gram matrix

Chapter D

Complete Python source

The following program generates every figure, every numerical table, and the complete built-in check ledger used in this paper.

```
from __future__ import annotations

import cmath
import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, FancyArrowPatch, FancyBboxPatch, Rectangle
import mpmath as mp
import numpy as np
import sympy as sp
from scipy.special import gamma as gamma_fn
from scipy.special import gammainc

mp.mp.dps = 60

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"
for directory in (FIGURES, RESULTS, TABLES):
    directory.mkdir(parents=True, exist_ok=True)

INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
```

```
RED = "#E76F51"
YELLOW = "#E9C46A"
PURPLE = "#6C5CE7"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW, PURPLE]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "font.size": 10.5,
        "axes.titlesize": 12.5,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)


def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.5, 6.0)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 11.5,
) -> None:
    patch = FancyBboxPatch(
        xy,
        width,
        height,
        boxstyle="round,pad=.012,rounding_size=.018",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.7,
    )
    ax.add_patch(patch)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=fontsize)


def arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
```

```
    text_offset: tuple[float, float] = (0.0, 0.03),
) -> None:
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.6, color=color)
    ax.add_patch(patch)
    if text:
        ax.text(
            (start[0] + end[0]) / 2 + text_offset[0],
            (start[1] + end[1]) / 2 + text_offset[1],
            text,
            ha="center",
            va="center",
            fontsize=10.5,
            color=color,
        )


# ---------------------------------------------------------------------------
# Arithmetic and character helpers
# ---------------------------------------------------------------------------


def mobius_sieve(nmax: int) -> np.ndarray:
    mu = np.zeros(nmax + 1, dtype=np.int8)
    mu[1] = 1
    primes: list[int] = []
    composite = np.zeros(nmax + 1, dtype=bool)
    for i in range(2, nmax + 1):
        if not composite[i]:
            primes.append(i)
            mu[i] = -1
        for p in primes:
            ip = i * p
            if ip > nmax:
                break
            composite[ip] = True
            if i % p == 0:
                mu[ip] = 0
                break
            mu[ip] = -mu[i]
    return mu


def prime_factors(n: int) -> list[int]:
    factors: list[int] = []
    p = 2
    remaining = n
    while p * p <= remaining:
        if remaining % p == 0:
            factors.append(p)
            while remaining % p == 0:
                remaining //= p
        p += 1
    if remaining > 1:
        factors.append(remaining)
    return factors


def squarefree_coprime_density(q: int) -> float:
    value = 1.0 / float(mp.zeta(2))
    for p in prime_factors(q):
        value *= p / (p + 1)
    return value


def characters_mod5(nmax: int) -> np.ndarray:
    """Return the four Dirichlet characters modulo 5.

    Character j sends the primitive residue 2 to exp(2*pi*i*j/4).
    """
    log_map = {1: 0, 2: 1, 4: 2, 3: 3}
    chars = np.zeros((4, nmax + 1), dtype=np.complex128)
```

```
    for n in range(1, nmax + 1):
        residue = n % 5
        if residue == 0:
            continue
        exponent = log_map[residue]
        for j in range(4):
            chars[j, n] = np.exp(2j * np.pi * j * exponent / 4)
    return chars


def dirichlet_l_hurwitz(s: complex, character_values: Sequence[complex], q: int) -> complex:
    total = mp.mpc(0)
    for r in range(1, q + 1):
        chi = complex(character_values[r % q]) if len(character_values) == q else complex(character_values[r])
        if abs(chi) > 0:
            total += chi * mp.zeta(s, mp.mpf(r) / q)
    return complex(total / (mp.mpc(q) ** s))


# ---------------------------------------------------------------------------
# Bernoulli-Hurwitz packets and Stieltjes germs
# ---------------------------------------------------------------------------


def bernoulli_packet(z: complex, a: complex, modulus: int, residue: int) -> complex:
    if abs(z) < 1e-28:
        return 1.0 / modulus
    zz = mp.mpc(z)
    q = mp.mpc(a + residue) / modulus
    return complex(-zz * (mp.mpf(modulus) ** (zz - 1)) * mp.zeta(1 - zz, q))


def child_parameters(a: float, m: int, r: int, ell: int) -> tuple[int, list[mp.mpf]]:
    q_total = ell * m
    parameters = [mp.mpf(a + r + child * m) / q_total for child in range(ell)]
    return q_total, parameters


def stieltjes_packet(ell: int, m: int, r: int, a: float, j: int, order: int) -> complex:
    omega = mp.e ** (2j * mp.pi / ell)
    q_total, parameters = child_parameters(a, m, r, ell)
    total = mp.mpc(0)
    for child, parameter in enumerate(parameters):
        total += omega ** (-j * child) * mp.stieltjes(order, parameter)
    return complex(total / mp.sqrt(ell))


def detail_channel(z: complex, ell: int, m: int, r: int, a: float, j: int) -> complex:
    q_total = ell * m
    omega = cmath.exp(2j * math.pi / ell)
    values = [bernoulli_packet(z, a, q_total, r + child * m) for child in range(ell)]
    return sum(omega ** (-j * child) * values[child] for child in range(ell)) / math.sqrt(ell)


def detail_derivative_from_stieltjes(
    ell: int,
    m: int,
    r: int,
    a: float,
    j: int,
    derivative_order: int,
) -> complex:
    if j == 0:
        raise ValueError("The compact derivative formula below is for non-DC channels")
    q_total = ell * m
    log_q = math.log(q_total)
    total = 0j
    for order in range(derivative_order):
        g = stieltjes_packet(ell, m, r, a, j, order)
        total += math.comb(derivative_order - 1, order) * (log_q ** (derivative_order - 1 - order)) * g
    return -(derivative_order / q_total) * total
```

```
def detail_variance(children: Sequence[complex]) -> complex:
    values = np.asarray(children, dtype=np.complex128)
    return complex(np.sum(values * values) - np.sum(values) ** 2 / len(values))


def transformed_details(children: Sequence[complex]) -> np.ndarray:
    values = np.asarray(children, dtype=np.complex128)
    return np.fft.fft(values) / math.sqrt(len(values))


def bernoulli_children_exact(n: int, a: sp.Rational, ell: int, m: int = 1, r: int = 0) -> list[sp.Expr]:
    q_total = ell * m
    return [
        sp.simplify(q_total ** (n - 1) * sp.bernoulli(n, sp.Rational(a) + sp.Rational(r + child * m, q_total)))
        for child in range(ell)
    ]


def primitive_packet_resultant_exact(n: int, a: sp.Rational, ell: int, m: int = 1, r: int = 0) -> sp.Expr:
    x = sp.symbols("x")
    children = bernoulli_children_exact(n, a, ell, m, r)
    polynomial = sum(children[index] * x**index for index in range(ell))
    return sp.factor(sp.resultant(sp.cyclotomic_poly(ell, x), polynomial, x))


# ---------------------------------------------------------------------------
# Positive Mobius packet energy and resolvent
# ---------------------------------------------------------------------------


def packet_energy_values(
    k_values: Sequence[int],
    a: float,
    beta: float,
    nmax: int,
    q: int = 1,
    mu: np.ndarray | None = None,
) -> np.ndarray:
    if mu is None:
        mu = mobius_sieve(nmax)
    n = np.arange(2, nmax + 1, dtype=np.float64)
    support = (mu[2:] != 0).astype(np.float64)
    if q > 1:
        support *= (np.gcd(n.astype(np.int64), q) == 1).astype(np.float64)
    base = support * n ** (-2 * a)
    log_x = 2.0 * np.log1p(-n ** (-beta))
    results = []
    for k in k_values:
        results.append(float(np.dot(base, np.exp(k * log_x))))
    # n=1 contributes only at k=0.  It is omitted because the asymptotic theory and all tests use k>=1.
    return np.asarray(results)


def energy_asymptotic_constant(a: float, beta: float, q: int = 1) -> float:
    alpha = (2 * a - 1) / beta
    return squarefree_coprime_density(q) / beta * float(gamma_fn(alpha)) * (2.0 ** (-alpha))


def energy_asymptotic(k: np.ndarray | float, a: float, beta: float, q: int = 1) -> np.ndarray:
    alpha = (2 * a - 1) / beta
    constant = energy_asymptotic_constant(a, beta, q)
    return constant * np.asarray(k, dtype=float) ** (-alpha)


def finite_differences(values: np.ndarray, max_order: int) -> list[np.ndarray]:
    rows = [np.asarray(values, dtype=float)]
    current = np.asarray(values, dtype=float)
    for _ in range(max_order):
        current = np.diff(current)
```

```
        rows.append(current)
    return rows


def energy_resolvent(z: complex, a: float, beta: float, nmax: int, q: int, mu: np.ndarray) -> complex:
    n = np.arange(2, nmax + 1, dtype=np.float64)
    support = (mu[2:] != 0).astype(np.float64)
    if q > 1:
        support *= (np.gcd(n.astype(np.int64), q) == 1).astype(np.float64)
    weights = support * n ** (-2 * a)
    x = (1.0 - n ** (-beta)) ** 2
    return complex(np.sum(weights / (1.0 - z * x)))


def squarefree_dirichlet_ratio(s: float, q: int = 1) -> mp.mpf:
    value = mp.zeta(s) / mp.zeta(2 * s)
    for p in prime_factors(q):
        value /= 1 + mp.mpf(p) ** (-s)
    return value


def energy_from_zeta_ratio(k: int, a: float, beta: float, q: int = 1) -> mp.mpf:
    total = mp.mpf(0)
    for j in range(2 * k + 1):
        total += (-1) ** j * math.comb(2 * k, j) * squarefree_dirichlet_ratio(2 * a + beta * j, q)
    return total


# ---------------------------------------------------------------------------
# Gamma scaling and character decoherence
# ---------------------------------------------------------------------------


def gamma_weighted_sample(
    k: int,
    a: float,
    beta: float,
    nmax: int,
    q: int,
    mu: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    n = np.arange(2, nmax + 1, dtype=np.float64)
    support = (mu[2:] != 0).astype(np.float64)
    if q > 1:
        support *= (np.gcd(n.astype(np.int64), q) == 1).astype(np.float64)
    log_weight = -2 * a * np.log(n) + 2 * k * np.log1p(-n ** (-beta))
    log_weight = np.where(support > 0, log_weight, -np.inf)
    maximum = np.max(log_weight)
    weights = np.exp(log_weight - maximum)
    weights /= weights.sum()
    u = 2.0 * k / (n**beta)
    order = np.argsort(u)
    return u[order], weights[order]


def weighted_moments(values: np.ndarray, weights: np.ndarray, max_order: int) -> list[float]:
    return [float(np.dot(weights, values**order)) for order in range(1, max_order + 1)]


def weighted_quantile(values: np.ndarray, weights: np.ndarray, probabilities: Sequence[float]) -> np.ndarray:
    cumulative = np.cumsum(weights)
    return np.interp(np.asarray(probabilities), cumulative, values)


def ks_distance_to_gamma(values: np.ndarray, weights: np.ndarray, alpha: float) -> float:
    empirical = np.cumsum(weights)
    theoretical = gammainc(alpha, values)
    return float(np.max(np.abs(empirical - theoretical)))


def character_gram_matrix(
```

```
    k: int,
    a: float,
    beta: float,
    nmax: int,
    mu: np.ndarray,
    characters: np.ndarray,
) -> np.ndarray:
    n = np.arange(2, nmax + 1, dtype=np.float64)
    squarefree = (mu[2:] != 0).astype(np.float64)
    weights = squarefree * n ** (-2 * a) * np.exp(2 * k * np.log1p(-n ** (-beta)))
    char_slice = characters[:, 2 : nmax + 1]
    weighted = char_slice * np.sqrt(weights)[None, :]
    return weighted @ weighted.conj().T


def normalized_correlation(gram: np.ndarray) -> np.ndarray:
    diagonal = np.real(np.diag(gram))
    denom = np.sqrt(np.outer(diagonal, diagonal))
    return np.divide(gram, denom, out=np.zeros_like(gram), where=denom > 0)


# ---------------------------------------------------------------------------
# Figures and checks
# ---------------------------------------------------------------------------


@dataclass
class Check:
    name: str
    passed: bool
    detail: str


def generate_all() -> dict[str, object]:
    checks: list[Check] = []
    tables: dict[str, list[dict[str, object]]] = {}

    # Figure 01: conceptual pipeline.
    fig, ax = diagram_axes((13.0, 6.2))
    ax.text(0.5, 0.94, "From Bernoulli children to local arithmetic invariants", ha="center", fontsize=20, weight="
     bold")
    stages = [
        (0.04, r"Hurwitz children\n$b_\ell(z;a)$", BLUE),
        (0.23, "cyclotomic DFT\n$W_j(z;a)$", TEAL),
        (0.42, "Stieltjes germ\n$G_{j,m}(a)$", ORANGE),
        (0.61, r"scalar invariants\n$\mathcal{V}$ and $\mathcal{N}$", MAGENTA),
        (0.80, "positive energy\n$E(k)$ and resolvent", RED),
    ]
    for x0, text, color in stages:
        box(ax, (x0, 0.55), 0.15, 0.18, text, face=color + "20", edge=color, fontsize=12)
    for left, right in zip(stages[:-1], stages[1:], strict=True):
        arrow(ax, (left[0] + 0.15, 0.64), (right[0], 0.64), color=INK)
    box(ax, (0.20, 0.17), 0.22, 0.16, r"$L^{(m)}(1,\chi)$\ncharacter transform", face=TEAL + "18", edge=TEAL,
     fontsize=12)
    box(ax, (0.56, 0.17), 0.22, 0.16, "Gamma scaling\nand character decoherence", face=PURPLE + "18", edge=PURPLE,
     fontsize=12)
    arrow(ax, (0.50, 0.55), (0.31, 0.33), color=TEAL)
    arrow(ax, (0.88, 0.55), (0.67, 0.33), color=PURPLE)
    ax.text(0.5, 0.065, "Every map is exact at finite packet depth.  Asymptotic statements enter only in the energy
     and decoherence limits.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "01_theory_pipeline")

    # Stieltjes packet matrix for L=4.
    ell, m, r, a = 4, 1, 0, 1.0
    stieltjes_matrix = np.array(
        [[stieltjes_packet(ell, m, r, a, j, order) for order in range(6)] for j in range(ell)],
        dtype=np.complex128,
    )
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    im0 = axes[0].imshow(np.real(stieltjes_matrix), cmap="coolwarm", aspect="auto")
    axes[0].set_title("Real parts of $G_{j,m}(1)$")
```

```
    axes[0].set_xlabel("Stieltjes order $m$")
    axes[0].set_ylabel("cyclotomic channel $j$")
    axes[0].set_xticks(range(6))
    axes[0].set_yticks(range(ell))
    fig.colorbar(im0, ax=axes[0], fraction=0.045)
    im1 = axes[1].imshow(np.imag(stieltjes_matrix), cmap="coolwarm", aspect="auto")
    axes[1].set_title("Imaginary parts of $G_{j,m}(1)$")
    axes[1].set_xlabel("Stieltjes order $m$")
    axes[1].set_ylabel("cyclotomic channel $j$")
    axes[1].set_xticks(range(6))
    axes[1].set_yticks(range(ell))
    fig.colorbar(im1, ax=axes[1], fraction=0.045)
    fig.suptitle("Cyclotomic Stieltjes packets are the local Taylor data of the Bernoulli-Lerch detail bank",
     fontsize=16.5, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "02_stieltjes_packet_matrix")

    # Two-channel derivative example.
    derivative_exact = -math.log(2) / math.sqrt(2)
    derivative_formula = detail_derivative_from_stieltjes(2, 1, 0, 1.0, 1, 1)
    checks.append(Check("two_channel_first_derivative", abs(derivative_formula - derivative_exact) < 1e-14, f"error
     ={abs(derivative_formula-derivative_exact):.3e}"))
    z_values = np.linspace(-0.35, 0.35, 241)
    detail_values = np.array([detail_channel(float(z), 2, 1, 0, 1.0, 1).real for z in z_values])
    linear = derivative_exact * z_values
    fig, ax = plt.subplots(figsize=(11.8, 4.8))
    ax.plot(z_values, detail_values, lw=2.1, label="exact $W_1(z;1)$")
    ax.plot(z_values, linear, lw=1.7, linestyle="--", label=r"$-(\log 2)z/\sqrt{2}$")
    ax.axhline(0, color=GRID, lw=1)
    ax.axvline(0, color=GRID, lw=1)
    ax.set_xlabel("complex order restricted to real $z$")
    ax.set_ylabel("detail value")
    ax.set_title(r"The first Bernoulli-Euler detail coefficient is $-(\log 2)/\sqrt{2}$")
    ax.legend()
    ax.grid(alpha=0.18)
    fig.tight_layout()
    save_figure(fig, "03_two_channel_germ")

    # Derivative checks through order 4 by direct high-precision differentiation
    # of the Hurwitz-zeta child formula.
    def detail_mp(z_value: mp.mpf | mp.mpc) -> mp.mpc:
        z_mp = mp.mpc(z_value)
        values = []
        for child in range(2):
            parameter = mp.mpf(1 + child) / 2
            if abs(z_mp) < mp.mpf("1e-70"):
                child_value = mp.mpf(1) / 2
            else:
                child_value = -z_mp * (mp.mpf(2) ** (z_mp - 1)) * mp.zeta(1 - z_mp, parameter)
            values.append(child_value)
        return (values[0] - values[1]) / mp.sqrt(2)

    derivative_rows: list[dict[str, object]] = []
    for nder in range(1, 5):
        predicted = detail_derivative_from_stieltjes(2, 1, 0, 1.0, 1, nder)
        numeric_mp = mp.diff(detail_mp, mp.mpf(0), nder)
        numeric = complex(numeric_mp)
        error = abs(predicted - numeric)
        derivative_rows.append({"order": nder, "predicted": predicted.real, "numeric": numeric.real, "error": error
     })
        checks.append(Check(f"stieltjes_derivative_order_{nder}", error < 2e-14, f"error={error:.3e}"))
    tables["stieltjes_derivatives"] = derivative_rows

    # Character transform and L derivatives for nonprincipal character mod 4.
    q = 4
    chi_values = [mp.mpc(0), mp.mpc(1), mp.mpc(0), mp.mpc(-1), mp.mpc(0)]
    character_rows: list[dict[str, object]] = []
    for order in range(4):
        lhs_mp = sum(chi_values[r] * mp.stieltjes(order, mp.mpf(r) / q) for r in range(1, q + 1))
        lhs = complex(lhs_mp)
        rhs_sum = mp.mpc(0)
```

```
        for h in range(order + 1):
            derivative = mp.diff(
                lambda ss: mp.dirichlet(ss, [0, 1, 0, -1]),
                1,
                h,
            )
            rhs_sum += math.comb(order, h) * (mp.log(q) ** (order - h)) * derivative
        rhs = ((-1) ** order) * q * rhs_sum
        error = abs(lhs - complex(rhs))
        character_rows.append({"order": order, "packet_sum_real": lhs.real, "formula_real": float(mp.re(rhs)), "error": error})
        checks.append(Check(f"character_derivative_transform_{order}", error < 5e-8, f"error={error:.3e}"))
    tables["character_derivatives"] = character_rows
    fig, ax = plt.subplots(figsize=(11.8, 4.7))
    orders = np.arange(4)
    lhs_values = [row["packet_sum_real"] for row in character_rows]
    rhs_values = [row["formula_real"] for row in character_rows]
    width = 0.36
    ax.bar(orders - width / 2, lhs_values, width, label="Stieltjes packet sum")
    ax.bar(orders + width / 2, rhs_values, width, label="$L$-derivative formula")
    ax.set_xticks(orders, [f"$m={value}$" for value in orders])
    ax.set_ylabel("real value")
    ax.set_title(r"Character packets recover the derivatives of $L(s,\chi_4)$ at $s=1$")
    ax.grid(axis="y", alpha=0.18)
    ax.legend()
    fig.tight_layout()
    save_figure(fig, "04_character_L_derivatives")

    # Detail variance identities on a deterministic complex example.
    children_complex = np.array([1.1 + 0.2j, -0.4 + 0.7j, 0.8 - 0.5j, 0.3 + 0.1j])
    transformed = transformed_details(children_complex)
    variance_direct = detail_variance(children_complex)
    variance_fourier = sum(transformed[j] * transformed[-j % len(transformed)] for j in range(1, len(transformed)))
    pairwise = sum((children_complex[l] - children_complex[m]) ** 2 for l in range(4) for m in range(l + 1, 4)) / 4
    checks.append(Check("detail_variance_fourier_identity", abs(variance_direct - variance_fourier) < 1e-13, f"error={abs(variance_direct-variance_fourier):.3e}"))
    checks.append(Check("detail_variance_pairwise_identity", abs(variance_direct - pairwise) < 1e-13, f"error={abs(variance_direct-pairwise):.3e}"))
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    axes[0].scatter(children_complex.real, children_complex.imag, s=85)
    for index, value in enumerate(children_complex):
        axes[0].text(value.real + 0.03, value.imag + 0.03, f"$b_{index}$")
    axes[0].axhline(0, color=GRID, lw=1)
    axes[0].axvline(0, color=GRID, lw=1)
    axes[0].set_xlabel("real part")
    axes[0].set_ylabel("imaginary part")
    axes[0].set_title("child packets")
    axes[0].grid(alpha=0.15)
    axes[1].bar(np.arange(4), np.abs(transformed), color=COLORS[:4])
    axes[1].set_xticks(range(4), ["DC", "detail 1", "detail 2", "detail 3"])
    axes[1].set_ylabel("channel magnitude")
    axes[1].set_title("unitary child transform")
    axes[1].grid(axis="y", alpha=0.15)
    fig.suptitle("The holomorphic detail variance is the non-DC Fourier energy without conjugation", fontsize=16.2, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "05_detail_variance_identity")

    # Bernoulli-Euler variance.
    x_values = np.linspace(0, 1, 401)
    fig, ax = plt.subplots(figsize=(11.8, 4.9))
    for n_order, color in [(4, BLUE), (6, TEAL), (8, MAGENTA)]:
        function = sp.lambdify(sp.symbols("x"), sp.Rational(n_order**2, 8) * sp.euler(n_order - 1, sp.symbols("x")) ** 2, "numpy")
        ax.plot(x_values, function(x_values), lw=1.8, color=color, label=rf"$n={n_order}$")
    ax.set_xlabel("Bernoulli parameter $a$")
    ax.set_ylabel(r"$\mathcal{V}_2(n;a)$")
    ax.set_title("The two-channel refinement variance is an Euler-polynomial square")
    ax.legend()
    ax.grid(alpha=0.18)
    fig.tight_layout()
```

```
    save_figure(fig, "06_bernoulli_euler_variance")

    # Exact L=3 example.
    children_l3 = bernoulli_children_exact(4, sp.Rational(0), 3)
    variance_l3 = sp.simplify(sum(value**2 for value in children_l3) - sum(children_l3) ** 2 / 3)
    checks.append(Check("three_channel_variance_exact", variance_l3 == sp.Rational(32, 27), f"value={variance_l3}"))
    fig, ax = plt.subplots(figsize=(10.8, 4.8))
    positions = np.arange(3)
    values = [float(value) for value in children_l3]
    ax.bar(positions, values, color=[BLUE, TEAL, ORANGE])
    ax.axhline(float(sum(children_l3) / 3), color=INK, linestyle="--", label="parent mean")
    for position, value in zip(positions, children_l3, strict=True):
        ax.text(position, float(value) + (0.05 if float(value) >= 0 else -0.10), f"${sp.latex(value)}$", ha="center")
    ax.set_xticks(positions, ["child 0", "child 1", "child 2"])
    ax.set_ylabel("packet value")
    ax.set_title(r"Order four, three-channel Bernoulli refinement: $\mathcal{V}_3=32/27$")
    ax.legend()
    ax.grid(axis="y", alpha=0.16)
    fig.tight_layout()
    save_figure(fig, "07_three_channel_variance")

    # Primitive resultants for L=3 and 5.
    resultant_rows: list[dict[str, object]] = []
    for ell_value in (3, 4, 5):
        for n_order in range(2, 11):
            value = primitive_packet_resultant_exact(n_order, sp.Rational(0), ell_value)
            resultant_rows.append({"L": ell_value, "n": n_order, "value": str(value), "zero": bool(value == 0)})
            checks.append(Check(f"resultant_rational_L{ell_value}_n{n_order}", bool(value.is_rational), f"value={value}"))
    tables["resultants"] = resultant_rows
    fig, ax = plt.subplots(figsize=(12.2, 5.0))
    for ell_value, color in [(3, BLUE), (4, TEAL), (5, MAGENTA)]:
        subset = [row for row in resultant_rows if row["L"] == ell_value]
        n_orders = np.array([row["n"] for row in subset])
        magnitudes = np.array([math.log10(abs(float(sp.Rational(row["value"]))) + 1e-30) for row in subset])
        ax.plot(n_orders, magnitudes, marker="o", color=color, label=rf"$L={ell_value}$")
    ax.set_xlabel("integer order $n$")
    ax.set_ylabel(r"$\log_{10}|\mathcal{N}^{(L)}(n;0)|$")
    ax.set_title("Primitive packet resultants compress a full cyclotomic detail orbit into one scalar")
    ax.legend()
    ax.grid(alpha=0.18)
    fig.tight_layout()
    save_figure(fig, "08_primitive_resultants")

    # Energy computations.
    nmax = 500_000
    mu = mobius_sieve(nmax)
    a_energy = 2.0
    beta_energy = 2.0
    k_dense = np.arange(1, 161)
    energy_dense = packet_energy_values(k_dense, a_energy, beta_energy, nmax, 1, mu)
    differences = finite_differences(energy_dense, 8)
    sign_matrix = np.full((9, 120), np.nan)
    for order, row in enumerate(differences):
        usable = min(120, len(row))
        sign_matrix[order, :usable] = ((-1) ** order) * row[:usable]
    checks.append(Check("complete_monotonicity_numeric", bool(np.nanmin(sign_matrix) >= -2e-18), f"minimum={np.nanmin(sign_matrix):.3e}"))
    fig, ax = plt.subplots(figsize=(12.0, 5.1))
    image = ax.imshow(np.log10(np.maximum(sign_matrix, 1e-30)), cmap="viridis", aspect="auto")
    ax.set_yticks(range(9), [rf"$m={order}$" for order in range(9)])
    ax.set_xlabel("finite-difference location $k$")
    ax.set_title(r"Complete monotonicity: $(-1)^m\Delta^mE(k)\geq 0$")
    fig.colorbar(image, ax=ax, fraction=0.027, pad=0.02, label="log10 positive value")
    fig.tight_layout()
    save_figure(fig, "09_complete_monotonicity")

    # Hankel positivity.
    energy_hankel = packet_energy_values(np.arange(0, 25), a_energy, beta_energy, nmax, 1, mu)
    hankel_rows = []
```

```
    for size in range(2, 11):
        matrix = np.array([[energy_hankel[i + j] for j in range(size)] for i in range(size)])
        eigenvalues = np.linalg.eigvalsh(matrix)
        hankel_rows.append({"size": size, "minimum_eigenvalue": float(eigenvalues[0]), "maximum_eigenvalue": float(eigenvalues[-1])})
        checks.append(Check(f"hankel_positive_size_{size}", bool(eigenvalues[0] > -1e-14), f"min_eig={eigenvalues[0]:.3e}"))
    tables["hankel"] = hankel_rows
    fig, ax = plt.subplots(figsize=(11.8, 4.8))
    sizes = np.array([row["size"] for row in hankel_rows])
    minimums = np.array([max(row["minimum_eigenvalue"], 1e-30) for row in hankel_rows])
    maximums = np.array([row["maximum_eigenvalue"] for row in hankel_rows])
    ax.semilogy(sizes, minimums, marker="o", label="smallest eigenvalue")
    ax.semilogy(sizes, maximums, marker="o", label="largest eigenvalue")
    ax.set_xlabel("Hankel size")
    ax.set_ylabel("eigenvalue")
    ax.set_title("Finite packet-energy Hankel matrices are positive")
    ax.legend()
    ax.grid(alpha=0.18, which="both")
    fig.tight_layout()
    save_figure(fig, "10_hankel_positivity")

    # Resolvent over the complex plane.
    xs = np.linspace(-1.4, 1.35, 170)
    ys = np.linspace(-1.25, 1.25, 135)
    magnitude = np.zeros((len(ys), len(xs)))
    n_res = np.arange(2, 5001, dtype=np.float64)
    support_res = (mu[2:5001] != 0).astype(np.float64)
    weights_res = support_res * n_res ** (-2 * a_energy)
    x_res = (1.0 - n_res ** (-beta_energy)) ** 2
    for row, y in enumerate(ys):
        z_row = xs.astype(np.complex128) + 1j * y
        values = np.sum(weights_res[None, :] / (1.0 - z_row[:, None] * x_res[None, :]), axis=1)
        magnitude[row, :] = np.log10(np.maximum(np.abs(values), 1e-300))
        if abs(y) < 0.025:
            magnitude[row, xs >= 1.0] = np.nan
    fig, ax = plt.subplots(figsize=(11.8, 5.5))
    image = ax.imshow(magnitude, extent=[xs[0], xs[-1], ys[0], ys[-1]], origin="lower", cmap="magma", aspect="auto")
    ax.plot([1, xs[-1]], [0, 0], color=WHITE, lw=3.0, label=r"excluded cut $[1,\infty)$")
    ax.set_xlabel("real part of $z$")
    ax.set_ylabel("imaginary part of $z$")
    ax.set_title("The packet-energy resolvent is zero-free off its positive-real cut")
    ax.legend(loc="lower left", facecolor="#111111", edgecolor=WHITE, labelcolor=WHITE, framealpha=0.82)
    fig.colorbar(image, ax=ax, fraction=0.027, pad=0.02, label=r"$\log_{10}|\mathcal{E}(z)|$")
    fig.tight_layout()
    save_figure(fig, "11_zero_free_resolvent")

    # Direct versus zeta-ratio finite differences at modest k.
    ratio_rows: list[dict[str, object]] = []
    for k in range(1, 9):
        direct = packet_energy_values([k], a_energy, beta_energy, nmax, 1, mu)[0]
        exact = float(energy_from_zeta_ratio(k, a_energy, beta_energy, 1))
        error = abs(direct - exact)
        ratio_rows.append({"k": k, "direct": direct, "zeta_ratio": exact, "truncation_error": error})
        checks.append(Check(f"zeta_ratio_energy_{k}", error < 2e-14, f"error={error:.3e}"))
    tables["zeta_ratio"] = ratio_rows
    fig, ax = plt.subplots(figsize=(11.8, 4.8))
    kvals = np.array([row["k"] for row in ratio_rows])
    direct_values = np.array([row["direct"] for row in ratio_rows])
    ratio_values = np.array([row["zeta_ratio"] for row in ratio_rows])
    ax.semilogy(kvals, direct_values, marker="o", lw=2, label="direct squarefree sum")
    ax.semilogy(kvals, ratio_values, marker="x", lw=1.5, linestyle="--", label=r"finite difference of $\zeta(s)/\zeta(2s)$")
    ax.set_xlabel("difference order $k$")
    ax.set_ylabel("energy")
    ax.set_title("One positive packet energy has two exact arithmetic coordinate systems")
    ax.legend()
    ax.grid(alpha=0.18, which="both")
    fig.tight_layout()
    save_figure(fig, "12_zeta_ratio_coordinates")

```

```
    # Energy asymptotic.
    k_asym = np.array([100, 300, 1000, 3000, 10000, 30000, 100000])
    energy_asym_values = packet_energy_values(k_asym, a_energy, beta_energy, nmax, 1, mu)
    alpha = (2 * a_energy - 1) / beta_energy
    constant = energy_asymptotic_constant(a_energy, beta_energy, 1)
    normalized = energy_asym_values * k_asym**alpha
    energy_rows = []
    for k, value, norm in zip(k_asym, energy_asym_values, normalized, strict=True):
        energy_rows.append({"k": int(k), "energy": value, "scaled": norm, "relative_error": abs(norm / constant - 1)
     })
    tables["energy_asymptotic"] = energy_rows
    checks.append(Check("energy_asymptotic_final", abs(normalized[-1] / constant - 1) < 0.01, f"relative_error={abs(
     normalized[-1]/constant-1):.3e}"))
    fig, ax = plt.subplots(figsize=(11.8, 4.8))
    ax.semilogx(k_asym, normalized, marker="o", lw=2, label=r"$k^{3/2}E_k$")
    ax.axhline(constant, color=RED, linestyle="--", label="proved asymptotic constant")
    ax.set_xlabel("$k$")
    ax.set_ylabel("scaled energy")
    ax.set_title("The principal packet energy has the exact $k^{-3/2}$ scale")
    ax.legend()
    ax.grid(alpha=0.18)
    fig.tight_layout()
    save_figure(fig, "13_energy_asymptotic")

    # Gamma scaling distribution.
    gamma_rows: list[dict[str, object]] = []
    fig, ax = plt.subplots(figsize=(11.8, 5.0))
    u_grid = np.linspace(0, 8, 500)
    gamma_density = u_grid ** (alpha - 1) * np.exp(-u_grid) / gamma_fn(alpha)
    ax.plot(u_grid, gamma_density, color=INK, lw=2.4, label=rf"Gamma$(\alpha={alpha:.1f},1)$")
    for index, k in enumerate([300, 3000, 30000, 100000]):
        values, weights = gamma_weighted_sample(k, a_energy, beta_energy, nmax, 1, mu)
        bins = np.linspace(0, 8, 70)
        hist, edges = np.histogram(values, bins=bins, weights=weights, density=True)
        centers = (edges[:-1] + edges[1:]) / 2
        ax.plot(centers, hist, lw=1.35, color=COLORS[index], label=rf"$k={k}$")
        moments = weighted_moments(values, weights, 2)
        quantiles = weighted_quantile(values, weights, [0.1, 0.5, 0.9])
        ks = ks_distance_to_gamma(values, weights, alpha)
        gamma_rows.append({
            "k": k,
            "mean": moments[0],
            "variance": moments[1] - moments[0] ** 2,
            "q10": quantiles[0],
            "q50": quantiles[1],
            "q90": quantiles[2],
            "ks": ks,
        })
    tables["gamma_scaling"] = gamma_rows
    checks.append(Check("gamma_mean_final", abs(gamma_rows[-1]["mean"] - alpha) < 0.03, f"mean={gamma_rows[-1]['mean
     ']:.6f}"))
    checks.append(Check("gamma_variance_final", abs(gamma_rows[-1]["variance"] - alpha) < 0.06, f"variance={
     gamma_rows[-1]['variance']:.6f}"))
    ax.set_xlim(0, 8)
    ax.set_xlabel(r"$U_k=2k/n^2$")
    ax.set_ylabel("probability density")
    ax.set_title("The active squarefree packet window converges to a Gamma law")
    ax.legend(ncol=2)
    ax.grid(alpha=0.16)
    fig.tight_layout()
    save_figure(fig, "14_gamma_scaling")

    fig, ax = plt.subplots(figsize=(11.8, 4.8))
    gamma_k = np.array([row["k"] for row in gamma_rows])
    ks_values = np.array([row["ks"] for row in gamma_rows])
    ax.loglog(gamma_k, ks_values, marker="o", lw=2)
    ax.set_xlabel("$k$")
    ax.set_ylabel("weighted Kolmogorov distance")
    ax.set_title("Finite packet distributions approach the limiting Gamma profile")
    ax.grid(alpha=0.18, which="both")
    fig.tight_layout()
```

```
    save_figure(fig, "15_gamma_convergence")

    # Character Gram and decoherence.
    characters = characters_mod5(nmax)
    gram_small = character_gram_matrix(100, a_energy, beta_energy, nmax, mu, characters)
    gram_large = character_gram_matrix(100000, a_energy, beta_energy, nmax, mu, characters)
    corr_small = normalized_correlation(gram_small)
    corr_large = normalized_correlation(gram_large)
    fig = plt.figure(figsize=(12.4, 4.9))
    grid = fig.add_gridspec(1, 3, width_ratios=[1.0, 1.0, 0.055], wspace=0.34)
    axes = [fig.add_subplot(grid[0, 0]), fig.add_subplot(grid[0, 1])]
    cax = fig.add_subplot(grid[0, 2])
    for ax, matrix, title in [(axes[0], corr_small, "$k=100$"), (axes[1], corr_large, "$k=100000$")]:
        image = ax.imshow(np.abs(matrix), cmap="viridis", vmin=0, vmax=1)
        ax.set_xticks(range(4), [r"$\chi_0$", r"$\chi_1$", r"$\chi_2$", r"$\chi_3$"])
        ax.set_yticks(range(4), [r"$\chi_0$", r"$\chi_1$", r"$\chi_2$", r"$\chi_3$"])
        ax.set_title(title)
        for row in range(4):
            for col in range(4):
                value = abs(matrix[row, col])
                ax.text(col, row, f"{value:.3f}", ha="center", va="center", color=WHITE if value > 0.55 else INK,
     fontsize=8.7)
    fig.colorbar(image, cax=cax, label="normalized magnitude")
    fig.suptitle("Distinct Dirichlet-character packets become asymptotically orthogonal", fontsize=16.5, weight="
     bold")
    fig.subplots_adjust(left=0.07, right=0.96, bottom=0.13, top=0.82)
    save_figure(fig, "16_character_gram")

    k_char = np.array([100, 300, 1000, 3000, 10000, 30000, 100000])
    char_rows = []
    max_offdiag = []
    for k in k_char:
        gram = character_gram_matrix(int(k), a_energy, beta_energy, nmax, mu, characters)
        corr = normalized_correlation(gram)
        mask = ~np.eye(4, dtype=bool)
        maximum = float(np.max(np.abs(corr[mask])))
        max_offdiag.append(maximum)
        char_rows.append({"k": int(k), "max_offdiag": maximum})
    tables["character_decoherence"] = char_rows
    fig, ax = plt.subplots(figsize=(11.8, 4.8))
    ax.loglog(k_char, max_offdiag, marker="o", lw=2, label="largest measured off-diagonal correlation")
    reference_constant = max_offdiag[0] * (k_char[0] ** 0.25)
    ax.loglog(k_char, reference_constant * k_char ** (-0.25), linestyle="--", color=RED, label=r"reference slope $k
     ^{-1/4}$")
    ax.set_xlabel("$k$")
    ax.set_ylabel("maximum normalized correlation")
    ax.set_title("Character decoherence removes cross-sector interference unconditionally")
    ax.legend()
    ax.grid(alpha=0.18, which="both")
    fig.tight_layout()
    save_figure(fig, "17_character_decoherence")
    checks.append(Check("character_decoherence_decreases", max_offdiag[-1] < max_offdiag[0], f"first={max_offdiag
     [0]:.4f}, last={max_offdiag[-1]:.4f}"))

    # Quantum/logarithmic pipeline.
    fig, ax = diagram_axes((13.0, 6.2))
    ax.text(0.5, 0.94, "Positive packet energy and signed coherence in a logarithmic spectrum", ha="center",
     fontsize=20, weight="bold")
    box(ax, (0.04, 0.58), 0.17, 0.17, "number basis\n" + r"$|n\rangle$", face=BLUE + "20", edge=BLUE, fontsize=13)
    box(ax, (0.29, 0.58), 0.18, 0.17, "log Hamiltonian\n" + r"$H|n\rangle=\log n|n\rangle$", face=TEAL + "20", edge=
     TEAL, fontsize=12)
    box(ax, (0.55, 0.58), 0.18, 0.17, "scale filter\n" + r"$e^{-kK_\beta}$", face=ORANGE + "20", edge=ORANGE,
     fontsize=13)
    box(ax, (0.81, 0.58), 0.15, 0.17, "character sign\n" + r"$\mathcal{M}_\chi$", face=MAGENTA + "20", edge=MAGENTA,
      fontsize=12)
    arrow(ax, (0.21, 0.665), (0.29, 0.665))
    arrow(ax, (0.47, 0.665), (0.55, 0.665))
    arrow(ax, (0.73, 0.665), (0.81, 0.665))
    box(ax, (0.13, 0.20), 0.30, 0.18, "diagonal power\n" + r"$E(k)=\langle\psi|\mathcal{M}^2e^{-2kK}|\psi\rangle$",
     face=TEAL + "14", edge=TEAL, fontsize=12)
    box(ax, (0.57, 0.20), 0.30, 0.18, "signed amplitude\n" + r"$F_k(t)=\langle\phi|\mathcal{M}e^{-kK}e^{-itH}|\phi\
```

```
    rangle$", face=MAGENTA + "14", edge=MAGENTA, fontsize=12)
    arrow(ax, (0.63, 0.58), (0.28, 0.38), color=TEAL)
    arrow(ax, (0.88, 0.58), (0.72, 0.38), color=MAGENTA)
    ax.text(0.5, 0.075, "The positive energy is completely understood. The remaining zeta difficulty is the internal
      coherence of the principal Mobius sector.", ha="center", fontsize=11.3, color=MUTED)
    save_figure(fig, "18_quantum_packet_pipeline")

    # Status map.
    fig, ax = diagram_axes((13.0, 6.5))
    ax.text(0.5, 0.95, "What has been proved and what remains open", ha="center", fontsize=21, weight="bold")
    proved = [
        "Stieltjes germ formulas",
        r"$L^{(m)}(1,\chi)$ transforms",
        "detail variance",
        "primitive resultants",
        "complete monotonicity",
        "Hankel positivity",
        "zero-free resolvent",
        "Gamma scaling",
        "character decoherence",
    ]
    open_items = [
        "principal Mobius coherence",
        "uniform GRH coherence",
        "packet canonical systems",
        "global zero-location theorem",
    ]
    ax.add_patch(FancyBboxPatch((0.05, 0.13), 0.52, 0.72, boxstyle="round,pad=.015,rounding_size=.02", facecolor=
     TEAL + "14", edgecolor=TEAL, lw=2))
    ax.add_patch(FancyBboxPatch((0.62, 0.13), 0.33, 0.72, boxstyle="round,pad=.015,rounding_size=.02", facecolor=
     ORANGE + "14", edgecolor=ORANGE, lw=2))
    ax.text(0.31, 0.80, "proved in this paper", ha="center", fontsize=15, weight="bold", color=TEAL)
    ax.text(0.785, 0.80, "open", ha="center", fontsize=15, weight="bold", color=ORANGE)
    y = 0.72
    for item in proved:
        ax.text(0.10, y, f"PASS  {item}", ha="left", va="center", fontsize=11.3)
        y -= 0.065
    y = 0.68
    for item in open_items:
        ax.text(0.67, y, f"-  {item}", ha="left", va="center", fontsize=11.3)
        y -= 0.12
    ax.text(0.5, 0.055, "The positive and cross-character layers are now controlled. The principal signed sector
     still carries the RH-scale obstruction.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "19_status_map")

    # Family map.
    fig, ax = diagram_axes((13.0, 6.4))
    ax.text(0.5, 0.95, "The second special-function family generated by the calculus", ha="center", fontsize=20,
     weight="bold")
    nodes = [
        (0.08, 0.68, 0.20, 0.14, "Bernoulli-Hurwitz\nchildren", BLUE),
        (0.40, 0.68, 0.20, 0.14, "cyclotomic Stieltjes\npackets", TEAL),
        (0.72, 0.68, 0.20, 0.14, "packet resultants\nand variance", ORANGE),
        (0.08, 0.30, 0.20, 0.14, "squarefree zeta-ratio\nfinite differences", MAGENTA),
        (0.40, 0.30, 0.20, 0.14, "Hausdorff energy\nresolvent", RED),
        (0.72, 0.30, 0.20, 0.14, "Gamma scale law\nand character Gram", PURPLE),
    ]
    for x0, y0, width, height, text, color in nodes:
        box(ax, (x0, y0), width, height, text, face=color + "18", edge=color, fontsize=12)
    arrow(ax, (0.28, 0.75), (0.40, 0.75))
    arrow(ax, (0.60, 0.75), (0.72, 0.75))
    arrow(ax, (0.18, 0.68), (0.18, 0.44), color=MAGENTA)
    arrow(ax, (0.28, 0.37), (0.40, 0.37), color=RED)
    arrow(ax, (0.60, 0.37), (0.72, 0.37), color=PURPLE)
    arrow(ax, (0.82, 0.68), (0.82, 0.44), color=PURPLE)
    ax.text(0.5, 0.08, "The analytic and arithmetic branches meet through finite transforms, positive moments, and
     asymptotic packet orthogonality.", ha="center", fontsize=11.4, color=MUTED)
    save_figure(fig, "20_function_family_map")

    # Save tables.
    for name, rows in tables.items():
```

```
        if not rows:
            continue
        path = TABLES / f"{name}.csv"
        with path.open("w", newline="", encoding="utf-8") as handle:
            writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
            writer.writeheader()
            writer.writerows(rows)

    passed = sum(check.passed for check in checks)
    summary = {
        "title": "Projector-Generated Special Functions II",
        "checks": len(checks),
        "passed": passed,
        "failed": len(checks) - passed,
        "figures": 20,
        "stieltjes_first_derivative": derivative_formula.real,
        "three_channel_variance": str(variance_13),
        "energy_asymptotic_constant": constant,
        "energy_final_relative_error": energy_rows[-1]["relative_error"],
        "gamma_final_mean": gamma_rows[-1]["mean"],
        "gamma_final_variance": gamma_rows[-1]["variance"],
        "gamma_final_ks": gamma_rows[-1]["ks"],
        "character_correlation_first": max_offdiag[0],
        "character_correlation_last": max_offdiag[-1],
    }
    (RESULTS / "results.json").write_text(json.dumps(summary, indent=2, default=lambda value: value.item() if
     hasattr(value, "item") else str(value)), encoding="utf-8")
    with (RESULTS / "checks.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=["name", "passed", "detail"])
        writer.writeheader()
        for check in checks:
            writer.writerow({"name": check.name, "passed": check.passed, "detail": check.detail})

    if passed != len(checks):
        failures = [check for check in checks if not check.passed]
        raise RuntimeError(f"{len(failures)} checks failed: {failures}")
    return summary


if __name__ == "__main__":
    print(json.dumps(generate_all(), indent=2, default=lambda value: value.item() if hasattr(value, "item") else str
     (value)))
```

Chapter E

Reproduction

From the source directory, run

```
python build_all.py
```

The command regenerates the figures and tables, runs the tests, and compiles the paper three times.

Bibliography

[1] Komorebi P., *Projector-Generated Special Functions I: Polyphase Differential Lifts, Bernoulli–Lerch Wavelets, Cyclotomic Detail Functions, and Möbius Coherence*, 2026.

[2] Komorebi P., *Bernoulli Transfer-Block Reformulation of Zeta*, proof-development note, 2026.

[3] NIST Digital Library of Mathematical Functions, Chapter 24, Bernoulli and Euler Polynomials.

[4] NIST Digital Library of Mathematical Functions, Chapter 25, Zeta and Related Functions.

[5] L. Báez-Duarte, A sequential Riesz-like criterion for the Riemann hypothesis, *International Journal of Mathematics and Mathematical Sciences*, 2005.

[6] G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers*, Oxford University Press.

[7] S. Wei, Y. Zhai, Q. Lu et al., The Riemann Hypothesis manifested in dynamical quantum phase transitions, *Nature Communications*, accepted 18 June 2026, DOI 10.1038/s41467-026-74935-8.